# Chapter 0: How to Approach an ML System Design Case

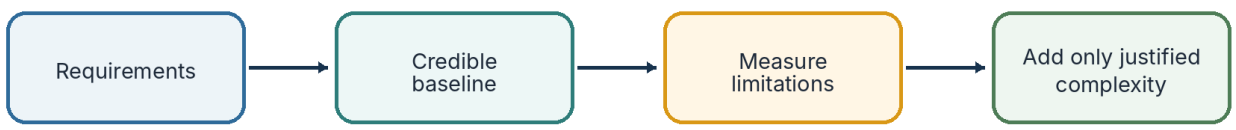
*A minimal-scope, alternatives-driven framework for interviews and internal design reviews*

PRIMARY REFERENCE

### Study foundation

This handbook began as personal study notes based largely on Machine Learning System Design Interview by Ali Aminian and Alex Xu. The restructuring, clarifications, comparison framework, diagrams, and interpretations are the author's own.

### Minimum sufficient design




A component belongs in the design only when it traces back to a requirement or a measured limitation.

**Requirement → design decision → expected benefit → metric or evidence**

*Figure 0.1 - Begin with the minimum sufficient design and add complexity only when evidence justifies it.*

CORE PRINCIPLE

### Every component needs a reason

Choose the smallest architecture that can satisfy the current requirements. Add granularity or complexity only to solve a specific product, quality, latency, scale, or operational limitation.

# 0. How to read this framework

The chapter separates source inspiration, confirmed principles, open choices, and starting recommendations.

FROM THE PRIMARY REFERENCE

### Source-derived study structure

The case-study topics and many design alternatives were learned from Machine Learning System Design Interview. Public materials should acknowledge the book and should not reproduce its text or figures verbatim.

CONFIRMED PRINCIPLE

### Minimal scope first

Related problems may be mentioned, but a separate objective should remain a separate service rather than being forced into one oversized architecture.

OPEN DESIGN DISCUSSION

### Context determines the answer

The framework does not declare one universal model. It asks readers to compare alternatives, state assumptions, and identify what evidence would change the decision.

STARTING RECOMMENDATION

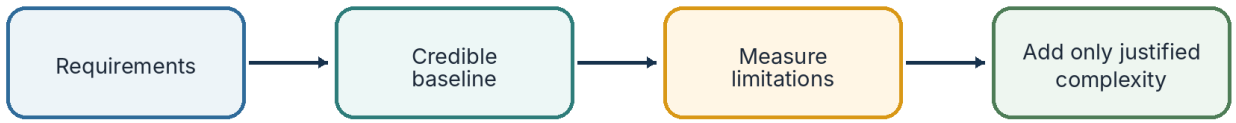
### Use a credible baseline

Begin with the easiest viable system that can meet the core task, then let measurable limitations justify the next layer of complexity.

# 1. Keep the scope minimal

Minimal does not mean weak; it means every component is traceable to a requirement.

### Minimum sufficient design




A component belongs in the design only when it traces back to a requirement or a measured limitation.

**Requirement → design decision → expected benefit → metric or evidence**

*Figure 1.1 - The minimum-sufficient-design loop.*

A minimal design may still include multiple stages, multimodal inputs, offline computation, human review, monitoring, and retraining. Those components belong only when the requirements need them.

| Question | Why it matters |
|---|---|
| What is the main service? | Prevents adjacent problems from silently expanding the case. |
| Which requirement justifies each component? | Makes the reasoning traceable and interview-ready. |
| What limitation is measured? | Prevents speculative complexity. |
| What would remove the component? | Encourages simpler production systems when quality is similar. |

**DESIGN DECISION**

**Main service:** _____

**Related but separate systems:** _____

**Out of scope:** _____

# 2. Use a recurring design order

A stable sequence keeps the discussion complete without forcing every case into the same architecture.

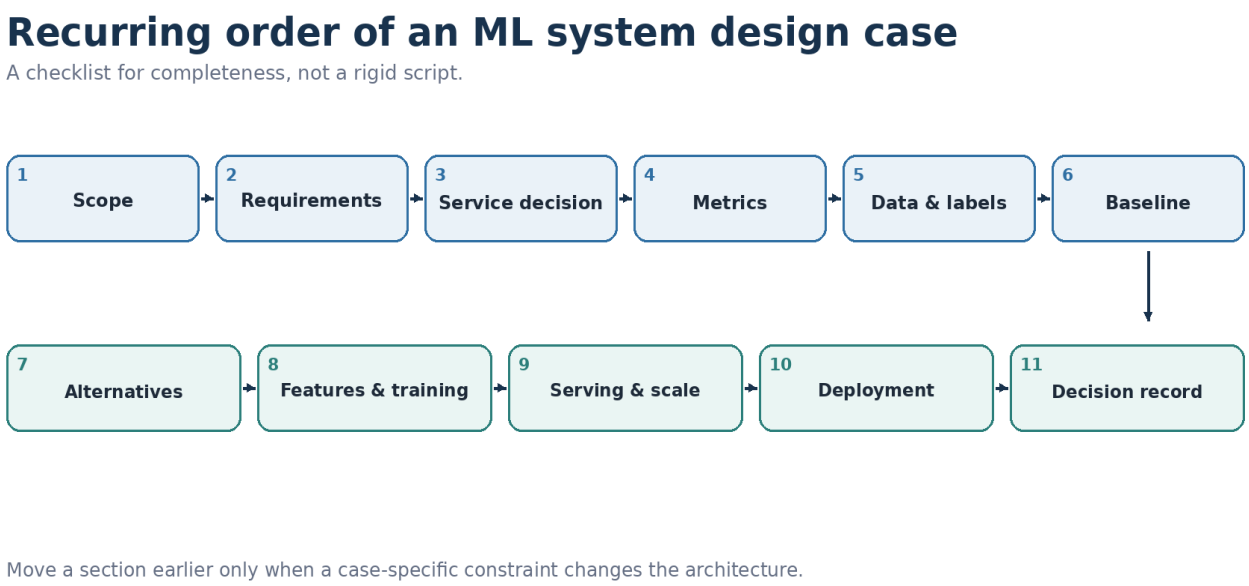



*Figure 2.1 - The recurring order used throughout the handbook.*

The sequence is a default. Candidate generation may appear before modeling in recommendation systems, while latency may be discussed earlier when it is the dominant constraint.

**STARTING RECOMMENDATION**

### Use the order as a checklist, not a script

Follow the structure until a case-specific constraint requires a different sequence. The objective is completeness and clarity, not rigid formatting.

# 3. Define scope, requirements, and the service decision

Select the model only after the input, output, product action, and constraints are explicit.

## Problem statement

A useful one-sentence objective is: Given X, predict or produce Y so the service can take Z action.

| Item | Questions to answer |
| --- | --- |
| Input | Who or what receives the prediction? What data is available at request time? |
| Model output | Score, label, embedding, ranked list, or generated response? |
| Service action | Recommend, remove, alert, approve, route, or review? |
| Boundary | Which related systems remain separate? |

## Functional and non-functional requirements

| Functional | Non-functional |
| --- | --- |
| What the system must do | Latency and throughput |
| Supported modalities or languages | Freshness and scale |
| Granularity of outputs | Reliability and fallback |
| Human-review or explanation needs | Cost, privacy, and maintainability |

## Separate the model output from the service action


**Examples**

- category scores → harmful score → keep / review / remove
- candidate scores → ranking policy → ordered recommendation list
- event probability → threshold policy → alert / no alert

*Figure 3.1 - Internal model granularity and the final product action are separate design layers.*

**DESIGN DECISION**

**Input:** _____

**Model output:** _____

**Service action:** _____

**Architecture-changing requirement:** _____

# 4. Connect product metrics to offline evaluation

Begin with the business objective, then choose an offline metric that approximates it.

| Layer | Example question | Common failure |
|---|---|---|
| Business objective | What user or business outcome should improve? | Optimizing a proxy with no product value |
| Online metric | What numerator, denominator, unit, and time window define success? | Metric growth caused by lower-quality behavior |
| Offline metric | What can be measured before deployment? | High offline score with weak product lift |
| Guardrail | What must not degrade? | Improvement achieved through harmful side effects |

CONFIRMED PRINCIPLE

### Metrics are part of the architecture

Rare positives may favor PR-AUC; ranked outputs may require Precision@K; probability-driven policies may require calibration. The metric should follow the task rather than habit.

DESIGN DECISION

**Primary product metric:** _____

**Offline proxy:** _____

**Guardrails:** _____

**Time window and evaluation unit:** _____

# 5. Define data and labels before the model

State exactly what one training example represents and enforce the prediction-time cutoff.

| Design item | Required decision |
|---|---|
| Entities | Users, items, posts, queries, interactions, graphs, reports, timestamps |
| Training unit | User-candidate pair, query-item pair, post-category pair, or event window |
| Labels | Positive, negative, unlabeled, weak, trusted, delayed, or multi-label |
| Time boundary | Only features available at or before prediction time |
| Missingness | Unknown, intentionally omitted, structurally absent, or delayed |

MINIMAL-SCOPE RULE

**Do not invent labels the source does not provide**

When negatives, attribution windows, or trusted labels are missing, preserve them as open design questions rather than silently filling the gap.

DESIGN DECISION

**Training unit:** _____

**Positive label:** _____

**Negative or unlabeled case:** _____

**Trusted benchmark:** _____

**Temporal cutoff:** _____

# 6. Compare alternatives through three recurring perspectives

The lenses help organize trade-offs without forcing exactly three separate models.

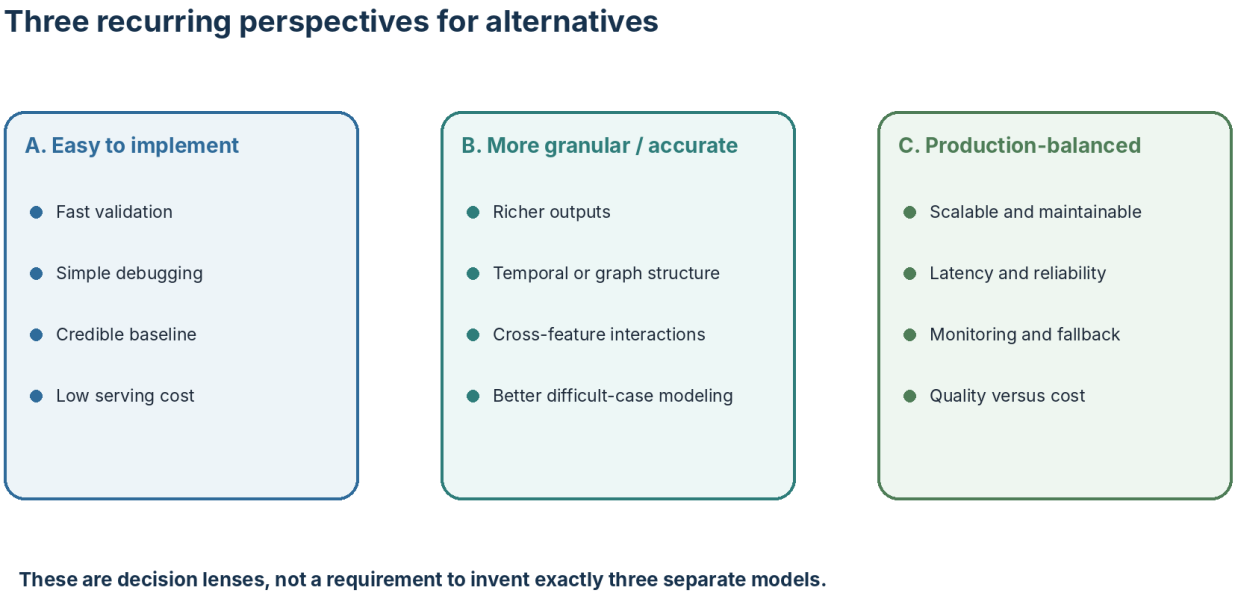



*Figure 6.1 - Easy baseline, richer modeling, and production-balanced design.*

| Perspective | Primary value | Typical cost |
|---|---|---|
| Easy to implement | Fast validation, debugging, and a credible reference | May omit granularity or complex relationships |
| More granular / accurate | Richer outputs, temporal structure, graph or cross-modal interactions | More data, tuning, compute, and debugging |
| Industrial preference | Quality balanced with latency, scale, reliability, and maintainability | May deliberately sacrifice peak offline accuracy |

CONFIRMED PRINCIPLE

**Industrial preference is not automatically the richest model**

A production team may keep the baseline, combine simple and rich stages, or run the rich model offline when the quality gain does not justify online complexity.

# 7. Give every alternative a decision rule

A list of models is not a design discussion until the switching conditions are explicit.

### How to compare an alternative

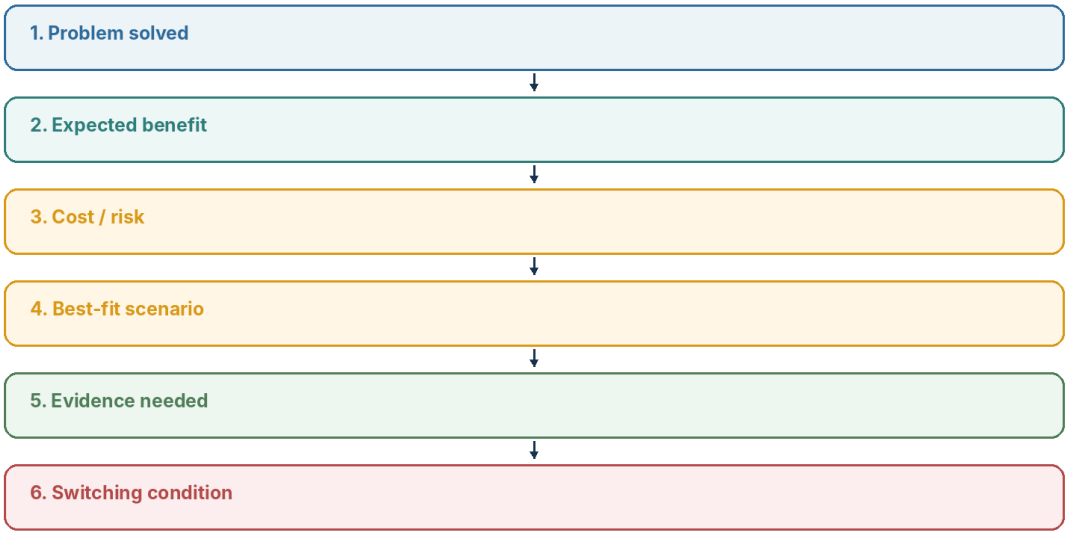



*Figure 7.1 - Six questions that turn an option list into a decision.*

| Option | Problem solved | Benefit | Cost / switch condition |
|---|---|---|---|
| Baseline | Basic task and measurement | Fast, reliable reference | Replace only after a measured failure |
| Richer model | Specific missed relationship or granularity | Potential quality lift | Keep only if lift justifies data and serving cost |
| Production-balanced | Operational constraints | Reliable user-facing system | Simplify when quality remains similar |

**DESIGN DECISION**

**Selected option:** _____

**Main reason:** _____

**Accepted trade-off:** _____

**Evidence needed:** _____

**Condition that changes the decision:** _____

# 8. Design features and training around observed problems

Use only features and optimization methods that solve a visible requirement or failure mode.

### Feature questions

| Question | Reason |
|---|---|
| Available at prediction time? | Prevents leakage and impossible online behavior |
| Stable and retrievable? | Prevents training-serving skew |
| Bias or privacy risk? | Separates predictive value from unacceptable harm |
| Needed in the main model? | Some features belong only in routing, review, or monitoring |

### Training method follows the failure mode

| Observed problem | Possible design response |
|---|---|
| Rare positive events | Imbalance-aware metric or loss |
| Enormous pair space | Negative sampling or candidate narrowing |
| Tasks dominate shared layers | Task weighting or gradient blending |
| Weak or noisy labels | Trusted benchmark, filtering, or confidence weighting |
| Probabilities drive policy | Calibration and category-specific thresholds |

STARTING RECOMMENDATION

**Start with the simplest loss and split**

Add focal loss, class balancing, gradient blending, or specialized sampling only when the data shows the corresponding failure.

# 9. Design serving and scale from the bottleneck

Separate offline and online work, then use the cheapest stage that preserves the required quality.

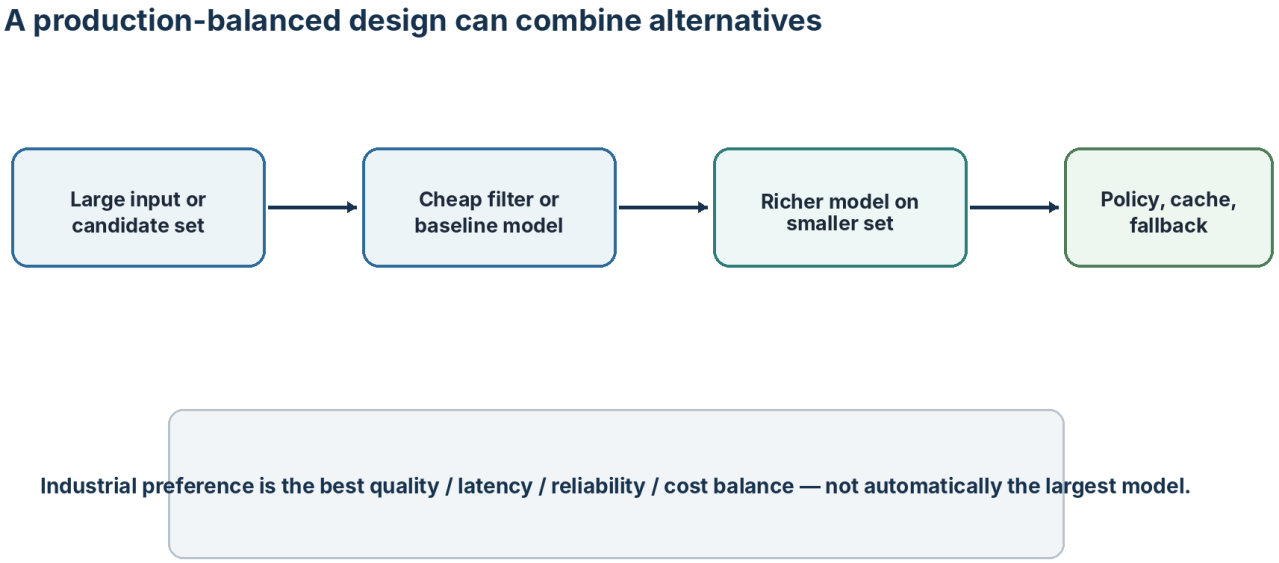



*Figure 9.1 - Simple and rich alternatives can be combined into a staged production design.*

| Serving pattern | Best when | Main trade-off |
|---|---|---|
| Fully online | Freshness is critical and the model is inexpensive | Highest request-time cost |
| Precomputed | Outputs change slowly and low latency matters | Staleness |
| Hybrid | Cached candidates or embeddings need fresh re-ranking | More moving parts, balanced latency |

| Measured bottleneck | Targeted technique |
|---|---|
| All-pairs search | Candidate narrowing or approximate retrieval |
| Long sequence | Approximate or linear attention |
| Slow online model | Precompute, cache, or lightweight re-ranking |
| Throughput | Batching or asynchronous processing |
| Review capacity | Confidence routing and prioritization |

# 10. Validate production risk before broad rollout

Choose shadow deployment, interleaving, or A/B testing according to the question and error cost.

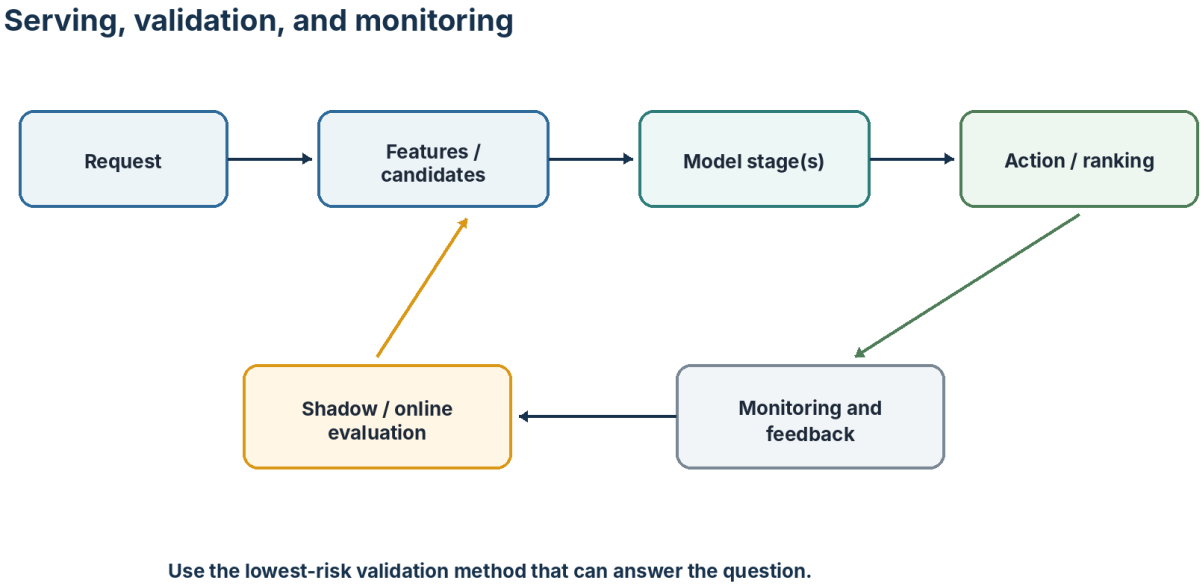



*Figure 10.1 - Serving connects to validation, monitoring, and feedback.*

| Method | Best use | Limitation |
|---|---|---|
| Shadow deployment | Safety, latency, score distribution, disagreement review | Cannot measure behavior after real actions |
| Interleaving | Fast comparison between ranking orders | Narrower than full product impact |
| A/B testing | Broader causal product impact | Riskier when errors directly affect users |

## Monitoring and refresh

- Input and feature drift
- Score distributions and calibration
- Category- or segment-level quality
- Latency, errors, and feature availability
- Business outcomes, reports, appeals, or user actions
- Retraining or refresh triggers and rollback conditions

**DESIGN DECISION**

**First validation method:** _____

**Monitoring metrics:** _____

**Exit criteria:** _____

**Rollback condition:** _____

# 11. Use a concise interview flow

A complete answer can remain structured even when time is limited.

| Order | Interview move |
|---|---|
| 1 | Clarify scope, input, output, and product action |
| 2 | State scale, latency, and freshness assumptions |
| 3 | Choose product and offline metrics |
| 4 | Define data, labels, and temporal cutoff |
| 5 | Establish the easiest viable baseline |
| 6 | Propose the richer option and the limitation it solves |
| 7 | Explain the production-balanced design |
| 8 | Describe serving, bottleneck, and fallback |
| 9 | Close with deployment, trade-offs, and switching conditions |

**INTERVIEW-READY STATEMENT**

### A defensible default

I would begin with the smallest system that satisfies the core requirements, establish a measurable baseline, and add complexity only when a specific product, quality, latency, or scaling limitation justifies it.

# 12. Avoid common design mistakes

The framework is designed to prevent impressive-sounding but weakly justified architectures.

| Mistake | Better reasoning |
| --- | --- |
| Start with the model | Start with the service decision and requirements |
| Put every related problem in scope | State the boundary and design separate services |
| List alternatives without a rule | Include benefit, cost, evidence, and switching condition |
| Assume highest accuracy is industrial preference | Balance quality with latency, reliability, and maintenance |
| Add scaling methods speculatively | Identify the measured bottleneck first |
| Confuse model output and product action | Add an explicit aggregation or policy layer |
| Use future data | Enforce prediction-time feature cutoffs |

# 13. Final case checklist

Use this one-page review before publishing or presenting a case.

| Area | Check |
|---|---|
| Scope | Main service, input, output, action, and out-of-scope items are explicit |
| Requirements | Functional and non-functional constraints are challengeable |
| Metrics | Product metric, offline proxy, guardrails, unit, and time window are clear |
| Data | Training unit, labels, trusted benchmark, missingness, and time cutoff are defined |
| Alternatives | Credible baseline, richer option, and production balance are compared |
| Training | Each method solves an observed problem |
| Serving | Offline/online split, bottleneck, cache, and fallback are explicit |
| Deployment | Validation, monitoring, refresh, and rollback are defined |
| Decision | Trade-offs, assumptions, evidence, and switching conditions are recorded |

**DESIGN DECISION**

**Chosen design:** _____

**Why it is minimum sufficient:** _____

**Accepted trade-offs:** _____

**Evidence still needed:** _____

**Conditions that change the design:** _____

# 14. Reference and attribution

Public chapters should clearly distinguish the primary study source from original transformations and commentary.

PRIMARY REFERENCE

## Machine Learning System Design Interview

Ali Aminian and Alex Xu. The book is the main study foundation for the case topics and many design alternatives in this handbook.

### Recommended public attribution

This material began as personal study notes based largely on Machine Learning System Design Interview by Ali Aminian and Alex Xu. The restructuring, clarifications, trade-off discussions, diagrams, interactive questions, and additional interpretations are my own.

This project is not affiliated with the authors or publisher and is not a substitute for the original book.

### Publication boundary

- Do not publish handwritten source scans.
- Do not reproduce book text, tables, or figures verbatim.
- Use original diagrams and wording for transformed explanations.
- Keep source-derived structure, confirmed interpretation, and added discussion visually distinct.

# Chapter 1
# Designing a Large-Scale Social Feed Ranking System

*A visual, alternatives-driven ML system design case study*

PRIMARY REFERENCE

**Study foundation**

This chapter began as personal study notes based largely on Machine Learning System Design Interview by Ali Aminian and Alex Xu. The restructuring, clarifications, trade-off discussions, diagrams, and interpretations are the author's own.

## Chapter at a glance

| Design area | Current direction |
|---|---|
| Scale | 500M DAU working assumption; ~200 ms ranking request |
| Objective | Predict several user behaviors and construct a serving score |
| Retrieval | Two-tower / ANN, social graph, and other candidate sources |
| Final model | Shared-bottom multi-task DNN |
| Validation | Offline evaluation, A/B test, then canary rollout |
| Scope boundary | Advertising monetization excluded |

# 1. Case map and minimum scope

Design the smallest credible system that can rank a social feed under large-scale latency and reliability constraints. Separate retrieval from final ranking, and add model granularity only where the product objective needs it.

Figure 1.1 - Ranking-system design map.

CONFIRMED INTERPRETATION

### Scale assumptions are challengeable

The 500 million DAU and roughly 200 ms latency values are working assumptions used to force large-scale design reasoning. They are not claims about a specific company.

OPEN DESIGN DISCUSSION

### What is the minimum sufficient scope?

**Main product action:** _____

**Scale and latency assumption:** _____

**Explicitly out of scope:** _____

**Condition that expands scope:** _____

# 2. Product objective and extensible labels

A ranking system should begin with the behavior it is intended to improve. One impression may generate several Boolean outcomes, allowing the same training example to support multiple product objectives.

Figure 1.2 - One impression can generate several extensible labels.

| Label type | Example | Open product decision |
|---|---|---|
| Positive engagement | like, comment, share | Which behaviors represent meaningful value? |
| Timing | interaction within T | What attribution window is appropriate? |
| Dwell | dwell time at or above threshold | What threshold separates useful consumption? |
| Negative feedback | hide, block | How strongly should it reduce the serving score? |

FROM THE ORIGINAL NOTES

### Extensible label set

New product objectives can become new labels and, in a multi-task model, new heads. The notes use 1 and 0 as positive and negative outcomes for each selected label.

# 3. Metrics by stage

The notes survey metrics rather than selecting one universal answer. The correct metric depends on the system stage, label type, ranking depth, and product question.

Figure 1.3 - Stage-specific metric selection.

| Metric | Best use | Main caution |
|---|---|---|
| Precision@K | Visible top-K quality | Does not fully describe order inside K |
| Recall@K | Candidate-generation coverage | May look small when positives greatly exceed K |
| mAP | Binary relevance across relevant ranks | Treats relevance as binary |
| MRR | First relevant result matters most | Ignores later relevant results |
| nDCG@K | Graded relevance and ordering | Requires a gain definition |
| PR-AUC | Rare positive behavior | Not a ranked-list metric |
| ROC-AUC | Broad discrimination | May hide top-K product weakness |

STARTING RECOMMENDATION

### Use a metric suite

Use Recall@K for retrieval, nDCG@K or Precision@K for final ranking, PR-AUC for rare behavior heads, and online product metrics for the final decision.

# 4. Model alternatives and their roles

Collaborative filtering, content-based models, two-tower retrieval, and a multi-task ranker are not mutually exclusive competitors. They solve different parts of the system.

Figure 1.4 - Model alternatives arranged as a staged production design.

| Option | Best when | Main benefit | Main cost |
|---|---|---|---|
| Collaborative filtering | Interaction history is strong | Credible baseline | Cold start |
| Content-based | New users or content need support | Uses attributes and modality features | May miss community interaction patterns |
| Two-tower | Large-scale retrieval is required | Scalable independent encoding | Less expressive pair interaction |
| Shared-bottom multi-task | Final ranking needs granular behavior scores | Shared learning plus task specialization | Higher serving and tuning complexity |

CONFIRMED INTERPRETATION

### Two-tower is broader than collaborative filtering

A two-tower model using only user-ID and item-ID embeddings is CF-like. Adding demographics, history, text, metadata, and context makes it a richer hybrid architecture.

PERSONAL LEARNING QUESTION

### Shared-bottom versus Mixture of Experts

The drawn model resembles MoE only conceptually through shared and specialized computation. It is not an MoE because it has no expert routing or gating mechanism.

# 5. Shared-bottom multi-task ranking

The preferred final ranker shares a common DNN representation and uses smaller task-specific heads to predict several behaviors.

Figure 1.5 - Multi-task architecture and the distinction between training and serving weights.

| Feature group | Examples | Role |
|---|---|---|
| User | ID, location, language, history | Viewer representation |
| Content | text, image, video, topic, author | Post representation |
| Historical engagement | views, likes, comments, dwell | Longer-term preference |
| User-author relationship | friendship, duration, strength, mutual links | Relationship-specific relevance |
| Context | time, device, request state | Request-specific behavior |

CONFIRMED INTERPRETATION

### Relationship features are model inputs

Friendship is not a separate prediction head and is not manually added after model scoring. It is a family of user-author input features.

MINIMAL-SCOPE RULE

### Do not copy serving weights into the loss

Training weights λ_i balance learning and stay nonnegative. Serving weights w_i express product utility and may be negative for hide or block.

# 6. Data, features, and freshness

Document feature meaning and availability separately. Historical does not automatically mean batch, and aggregated does not automatically mean stale.

| Property | Meaning | Example |
|---|---|---|
| Structured | Categorical or numeric fields | age, language, IDs, timestamps |
| Unstructured | Text, image, audio, or video | BERT, ViT, ResNet, or CLIP embeddings |
| Aggregated | Summarizes multiple events | seven-day engagement rate |
| Delayed | Arrives after asynchronous processing | hourly recomputed historical feature |
| Online | Available at request time | device and current request context |

CONFIRMED INTERPRETATION

### Aggregated and delayed are different properties

A feature can be aggregated and delayed at the same time. Freshness should be documented independently from feature semantics.

# 7. Training and loss weighting

For task i, the total training objective is the weighted sum of head-specific losses: $L_{total} = \Sigma_i \lambda_i L_i$.

- Head-specific parameters receive gradients from their own task.
- Shared layers receive the weighted combined influence of all tasks.
- Loss weights may consider label frequency, loss magnitude, task difficulty, gradient interference, and final ranking quality.
- The dwell threshold and attribution windows remain product decisions.

OPEN DESIGN DISCUSSION

### There is no universal formula for $\lambda_i$

Business priority matters, but the training weights should also be validated through per-task and final-ranking performance.

# 8. Candidate generation and final ranking

At very large scale, the richer ranker should score only a manageable candidate set. Several retrieval sources can contribute candidates before one final ranking stage.

Figure 1.6 - Candidate generation, deduplication, feature hydration, and final ranking.

CONFIRMED INTERPRETATION

### Multiple retrieval hits are informative

The system should deduplicate by content ID but retain source indicators. Agreement between retrieval sources can itself be useful ranking evidence.

OPEN DESIGN DISCUSSION

### How should candidate budgets be allocated?

**ANN pool size:** _____

**Graph pool size:** _____

**Other source pool:** _____

**Deduplication and source-feature policy:** _____

# 9. Indexing service boundary

The indexing service converts candidate representations into searchable structures. It hides the exact retrieval technique behind a stable service interface.

Figure 1.7 - Indexing service and optional compression techniques.

| Component | Responsibility |
| --- | --- |
| Feature store | Reusable model features |
| Indexing service | Build and version searchable representations |
| Candidate-generation service | Query one or more indexes |
| Ranking service | Score enriched, deduplicated candidates |

FROM THE ORIGINAL NOTES

### Optional vector compression

Vector quantization and product quantization can reduce memory and accelerate approximate retrieval. They remain optional techniques inside the indexing service.

# 10. Online validation and deployment

Offline performance is necessary but not sufficient. The new model should be tested online, then deployed gradually with rollback available.

Figure 1.8 - A/B testing, canary rollout, monitoring, and retraining.

| Method | Question answered |
|---|---|
| Offline evaluation | Does the model satisfy model-quality gates? |
| A/B test | Does it improve online product metrics? |
| Canary rollout | Can it operate safely at increasing production scale? |
| Rollback | Can traffic return quickly to a stable system? |

CONFIRMED INTERPRETATION

### The 5% / 95% split is an A/B test

After experimental performance and robustness are demonstrated, the canary rollout is a separate staged deployment step.

OPEN DESIGN DISCUSSION

### What permits wider deployment?

**Offline gate:** _____

**A/B primary metric:** _____

**Canary health metrics:** _____

**Rollback condition:** _____

# 11. Monitoring and continual learning

Model quality can decline as user behavior, content supply, and platform conditions change. Monitoring should connect model quality, data quality, and system health.

| Monitor | Examples | Possible response |
|---|---|---|
| Model quality | ranking metrics, head metrics | investigate or retrain |
| Data drift | feature and label distributions | refresh data or features |
| Product behavior | engagement and satisfaction | review objectives or weights |
| System health | latency and availability | rollback or scale infrastructure |

FROM THE ORIGINAL NOTES

### Continual-learning options

Regularization, adapters or new layers, and replaying some older data may reduce catastrophic forgetting. Replaying older data can distort the current time distribution.

# 12. Reference architecture and decision summary

One defensible design under the current assumptions is a hybrid system: reusable features and indexes are prepared asynchronously, while online retrieval, merging, feature hydration, and multi-task ranking meet the request-time latency target.

Figure 1.9 - One defensible large-scale feed-ranking architecture.

STARTING RECOMMENDATION

### Hybrid retrieval plus multi-task ranking

Use two-tower or ANN retrieval and graph-based sources to form a candidate set, then apply the richer shared-bottom multi-task model to the merged and hydrated candidates.

| Design area | Starting choice | Switch when |
|---|---|---|
| Objective | Multi-behavior prediction | One scalar target is demonstrably sufficient |
| Retrieval | ANN / two-tower plus graph sources | A simpler source meets coverage and latency |
| Final model | Shared-bottom multi-task DNN | A simpler baseline performs similarly |
| Metrics | Stage-specific suite | One metric reliably represents all decisions |
| Indexing | ANN abstraction with optional VQ/PQ | Exact search fits scale and latency |
| Validation | A/B test, then canary | Product risk requires another method |

# Chapter 2

# Predicting Ad Clicks on a Social Media Platform

*A visual, alternatives-driven ML system design case study*

PRIMARY REFERENCE

### Study foundation

This chapter began as personal study notes based largely on Machine Learning System Design Interview by Ali Aminian and Alex Xu. The restructuring, clarifications, trade-off discussions, diagrams, and interpretations are the author's own.

## Chapter at a glance

| Design area | Current direction |
|---|---|
| Objective | Predict calibrated click probability for ranking and revenue decisions |
| Product guardrail | Frequency policy plus hide and block behavior |
| Label strategy | Clicks are positive; unclicked impressions require an explicit policy |
| Baseline | Logistic Regression |
| Richer models | DCN and DeepFM as experiment directions |
| Large-scale architecture | Two-tower or ANN retrieval plus richer final CTR ranking |
| Validation | Shadow, canary, A/B testing, and calibration checks |

# 1. Case map and minimum scope

Predict click probability for eligible ads and use that probability in ranking. Keep frequency control, calibration, and user-experience guardrails explicit.

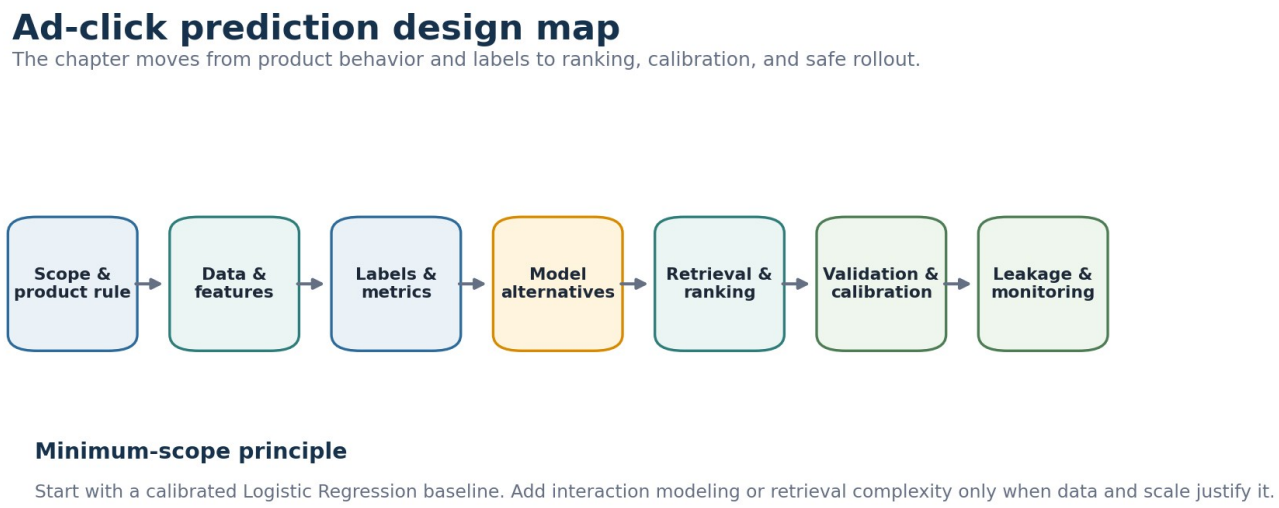



Figure 2.1 - Ad-click prediction design map.

CONFIRMED INTERPRETATION

## Binary label, probabilistic serving output

The training label is click or no click under the selected labeling policy, while the service consumes a probability used for ranking and revenue decisions.

OPEN DESIGN DISCUSSION

## What is the minimum sufficient scope?

Main product action: _____
Eligible inventory boundary: _____
Latency assumption: _____
Explicitly out of scope: _____

# 2. Product behavior and repeated exposure

The model can rank the same ad repeatedly, but product policy should decide when repetition becomes fatigue.

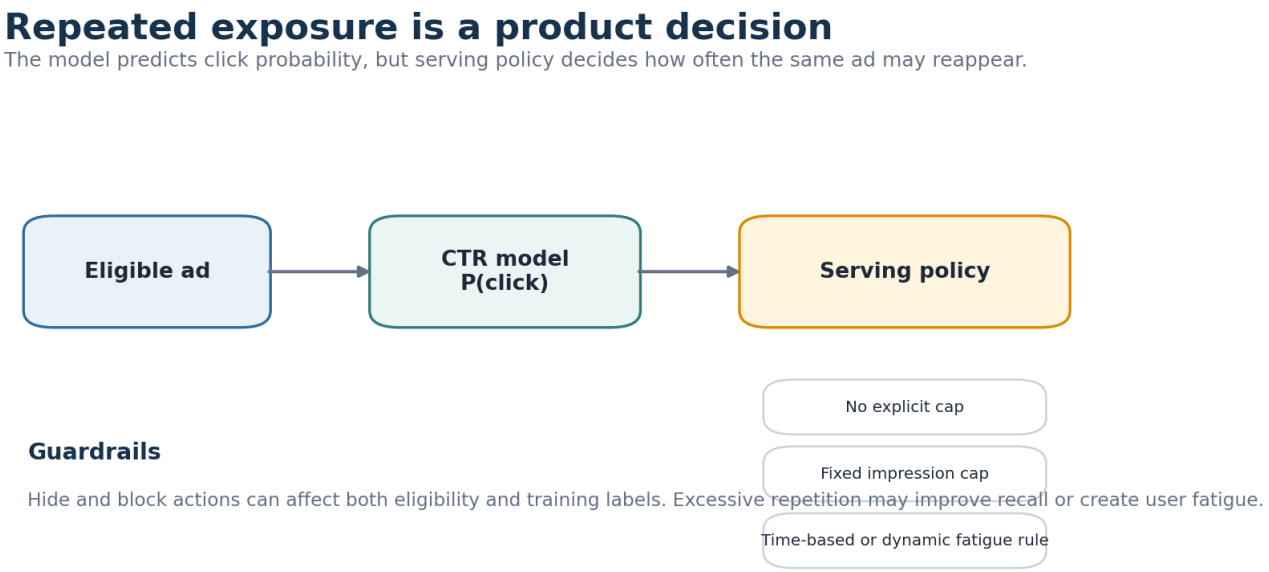



Figure 2.2 - Model prediction and repeated-exposure policy.

| Policy option | Main benefit | Main cost |
|---|---|---|
| No explicit cap | Simplest serving logic; may improve recall | Can create fatigue or a poor user experience |
| Fixed impression cap | Transparent and easy to enforce | One rule may not fit every user, ad, or campaign |
| Time-based fatigue period | Prevents short-term repetition | Requires choosing an arbitrary time window |
| Dynamic rule | Can respond to engagement and hide behavior | More modeling and policy complexity |

OPEN DESIGN DISCUSSION

### Frequency cap or fatigue period?

Selected policy: _____
Evidence supporting it: _____
User-experience guardrail: _____
Condition that changes the policy: _____

# 3. Data, features, and uncertain negatives

Clicks provide clear positives. Viewed-but-unclicked impressions may be true negatives, missed opportunities, or unlabeled observations.

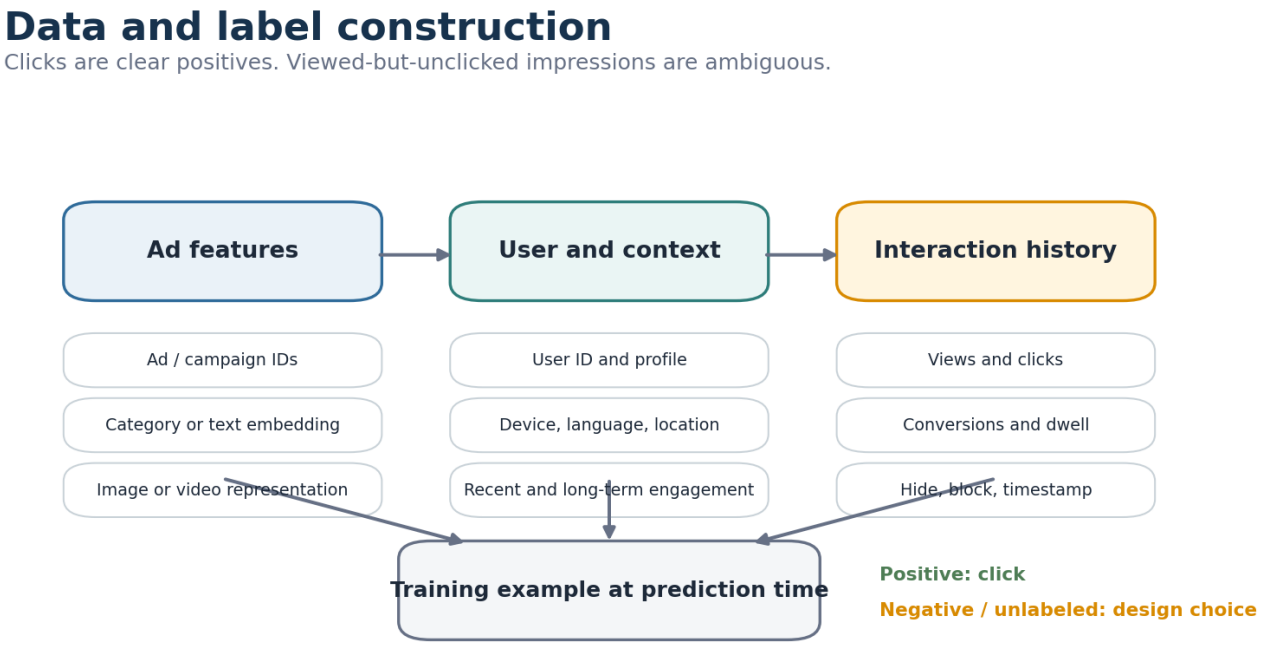



Figure 2.3 - Data families and label construction.

| Labeling direction | Benefit | Risk |
|---|---|---|
| Every unclicked view is negative | Simple and scalable | Introduces label noise and hidden positives |
| Hide and block are stronger negatives | Uses explicit negative intent | Sparse and behavior-dependent |
| Unclicked views remain unlabeled | Represents uncertainty | Needs more complex estimation |
| Weighted or PU approach | May model hidden positives | Requires assumptions and class-prior estimation |

STARTING RECOMMENDATION

### Begin with a transparent labeling baseline

Use clicks as positives and a clearly defined impression-based negative policy as the first baseline. Compare weighted or PU-style methods only when analysis shows hidden positives materially limit quality.

# 4. Evaluation metrics and baselines

Use probability, discrimination, business, and user-experience metrics together. Name the baseline behind every normalized metric.

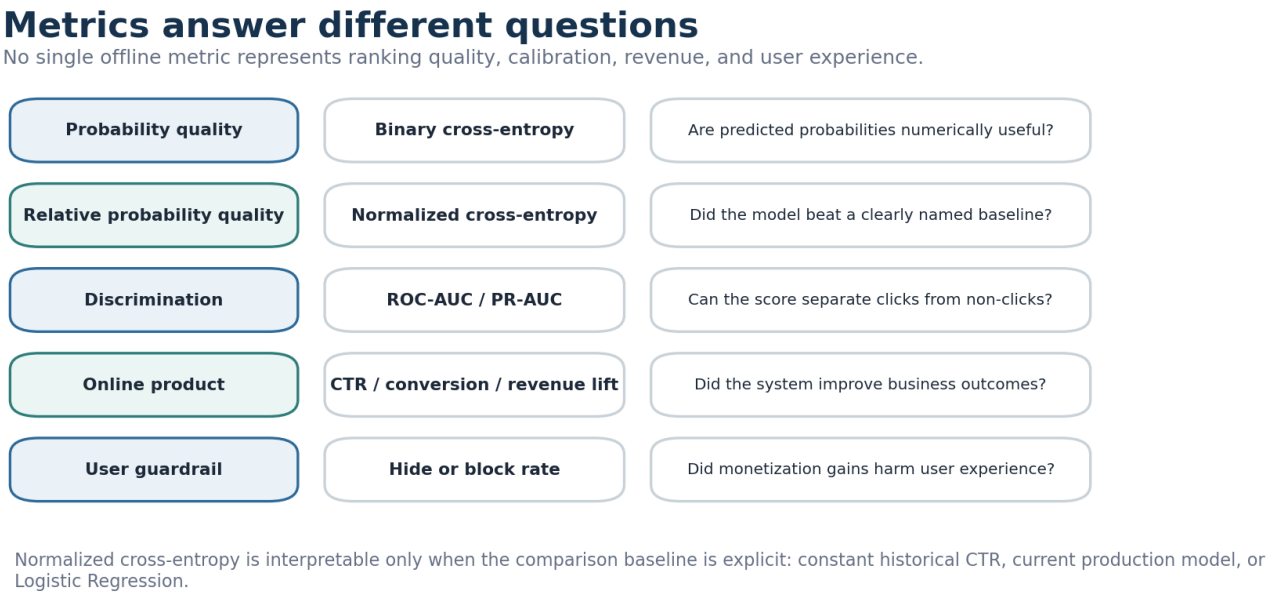



Figure 2.4 - Metric families and their decisions.

| Normalized CE baseline | Question answered | Limitation |
|---|---|---|
| Constant historical CTR | Did the model beat a naive prior? | Does not represent a learned production alternative |
| Current production model | Did the candidate improve over deployment? | Changes as production changes |
| Logistic Regression | Did richer modeling beat a simple learned baseline? | May not reflect current serving quality |

**OPEN DESIGN DISCUSSION**

## Which metric gates online testing?

Primary offline metric: _____
Normalized CE baseline: _____
Calibration check: _____
Online success metric: _____
User guardrail: _____

# 5. Easy-to-implement and structured baselines

Start with a model that is easy to debug, then add only the interaction capacity the data requires.

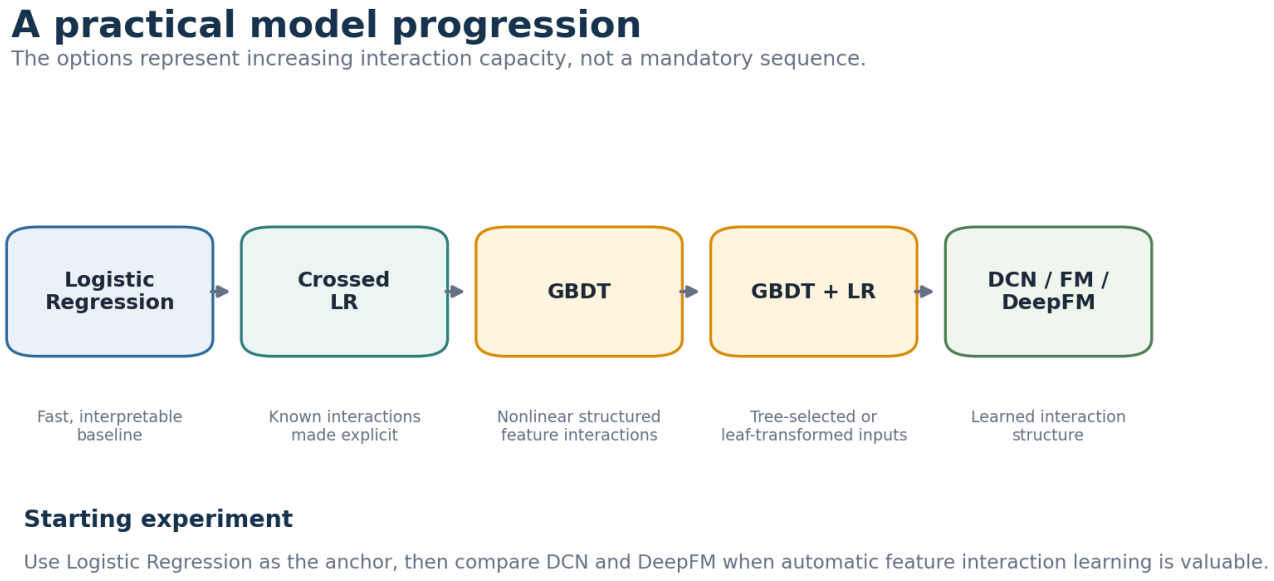



Figure 2.5 - Practical model progression.

| Approach | Best when | Main benefit | Main cost |
|---|---|---|---|
| Logistic Regression | A credible baseline is needed quickly | Fast, interpretable, easy to update | Manual interaction engineering |
| Feature crossing + LR | A few important interactions are already known | Keeps the final model simple | Sparse crosses and domain effort |
| GBDT | Structured features need nonlinear thresholds | Strong interaction discovery | Sparse IDs and continual updates can be awkward |
| GBDT + LR | Tree structure should feed a linear scorer | Flexible feature-selection or leaf-ID route | Two-stage pipeline complexity |

FROM THE ORIGINAL NOTES

### Two distinct GBDT + LR routes

One route uses importance or SHAP to select original features for LR. The other transforms examples into tree-leaf IDs and trains LR on the resulting representation. They remain alternatives rather than one mandatory procedure.

# 6. Richer interaction models

DCN, FM, and DeepFM differ mainly in how they represent interactions between sparse IDs, dense context, and content features.

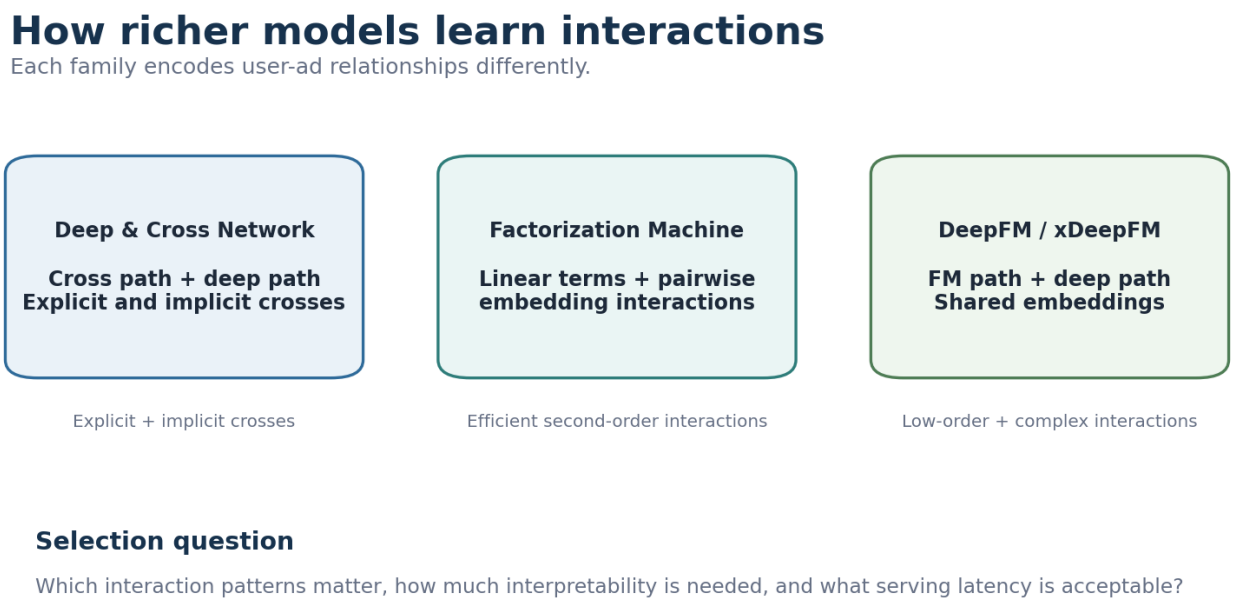



Figure 2.6 - Interaction-learning alternatives.

| Model | Interaction structure | Best fit |
|---|---|---|
| Deep & Cross Network | Explicit cross layers plus a deep network | Known and learned crosses are both valuable |
| Factorization Machine | Linear terms plus pairwise embedding interactions | Efficient second-order interactions dominate |
| DeepFM | FM path plus deep nonlinear path with shared embeddings | Low-order and complex interactions both matter |
| xDeepFM | Adds more explicit higher-order interaction learning | Higher-order structure provides measurable lift |

STARTING RECOMMENDATION

### Experiment rather than declare a universal winner

Use Logistic Regression as the anchor and compare DCN and DeepFM. Promote a richer model only when offline probability quality, calibration, latency, and online metrics justify the additional complexity.

# 7. Two-tower placement and candidate generation

The central system question is whether the two-tower model should perform final scoring or only reduce a large ad inventory to a manageable candidate set.

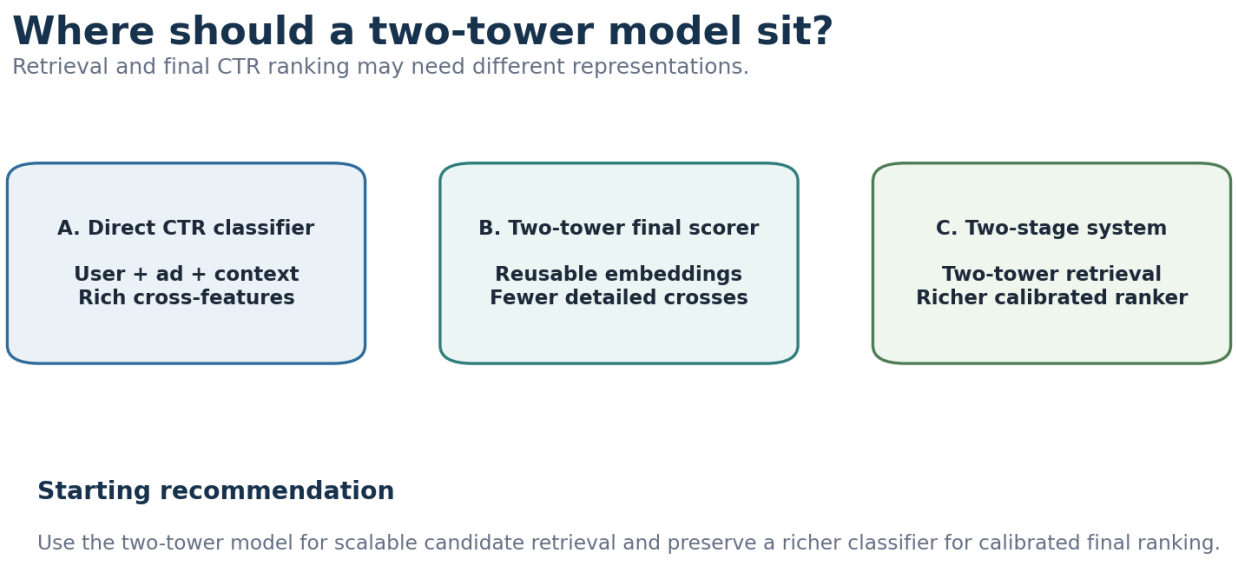



Figure 2.7 - Three placements for a two-tower model.

CONFIRMED INTERPRETATION

### Retrieval and final scoring solve different constraints

A reusable embedding model is attractive for large-scale retrieval, while the final CTR model may need detailed user-ad-context interactions and calibrated probabilities.

## Candidate generation and final ranking

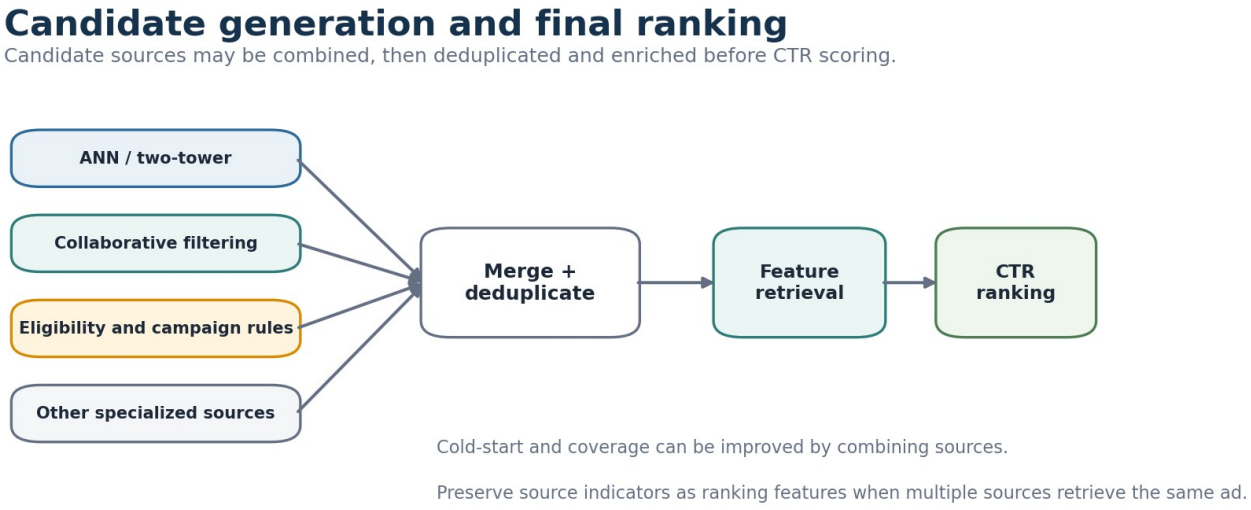



Figure 2.8 - Candidate sources, merge, feature hydration, and final CTR ranking.

STARTING RECOMMENDATION

### Use a two-stage system at large inventory scale

Retrieve candidates with reusable user and ad representations, merge and deduplicate multiple sources, then apply a richer calibrated CTR classifier that preserves cross-features.

# 8. Continual learning and drift response

User interests, active campaigns, inventory, and pricing conditions change. Updating should be triggered by observed drift or quality loss, not added automatically.

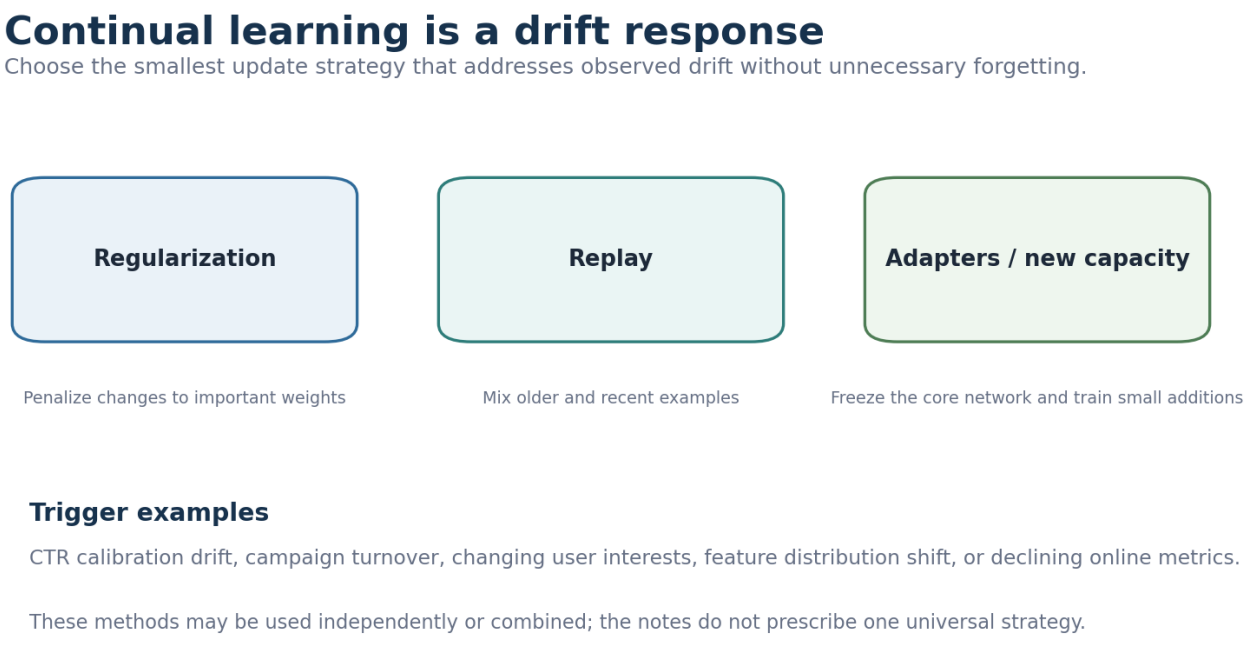



Figure 2.9 - Continual-learning alternatives.

| Method | Benefit | Risk |
|---|---|---|
| Regularization | Preserves important weights while adapting | May restrict useful changes |
| Replay | Retains older behaviors and reduces forgetting | Storage cost and distorted time distribution |
| Adapters or progressive capacity | Isolates updates from the core network | Growing model and operational complexity |
| Combination | Addresses several failure modes | More tuning and attribution difficulty |

OPEN DESIGN DISCUSSION

### What should trigger an update?

Primary drift signal: _____
Selected update method: _____
Retention risk: _____
Retraining threshold: _____
Rollback criterion: _____

# 9. Production validation and probability calibration

Shadow deployment validates operations; canary and A/B testing provide live behavior. Calibration determines whether scores can be used as trustworthy probabilities.

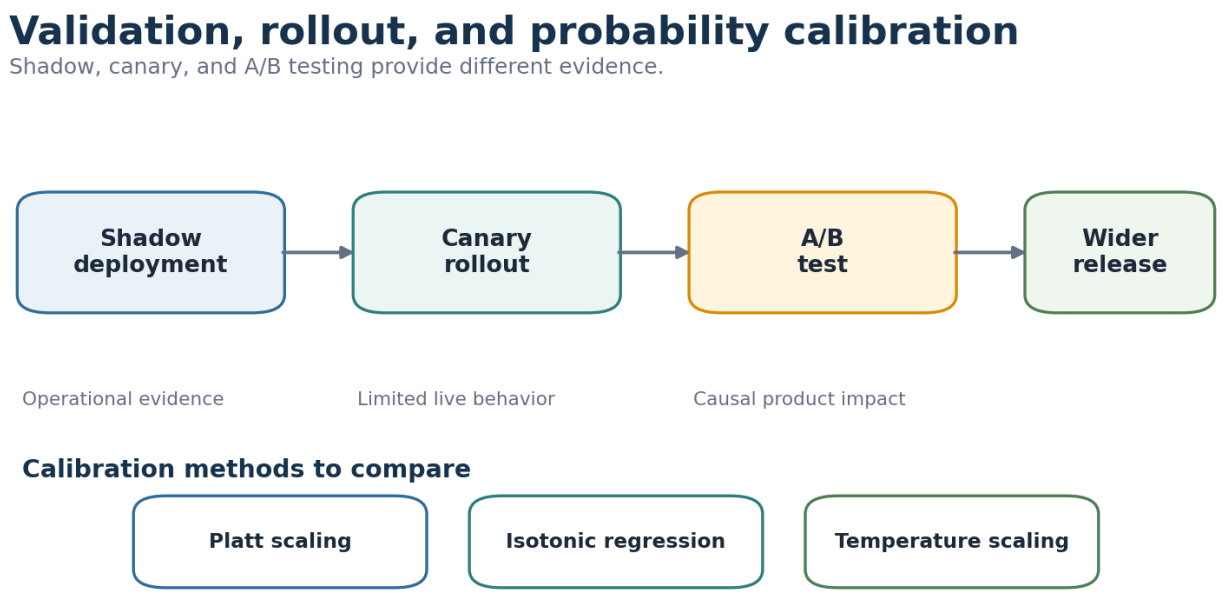



Figure 2.10 - Rollout evidence and calibration choices.

| Method | What it provides | What it cannot prove alone |
|---|---|---|
| Shadow deployment | Latency, errors, stability, score distribution | Actual click or revenue impact |
| Canary rollout | Limited live user and business behavior | Full-scale reliability |
| A/B test | Causal product and revenue comparison | Safe infrastructure ramp by itself |

| Calibration method | Strength | Constraint |
|---|---|---|
| Platt scaling | Simple parametric mapping | Assumes logistic shape |
| Isotonic regression | Flexible monotonic mapping | Needs enough calibration data |
| Temperature scaling | Simple score-sharpness adjustment | May be too limited for arbitrary miscalibration |

# 10. Time-aware training and leakage prevention

Features must reflect only information available at request time, and evaluation splits must preserve temporal order.

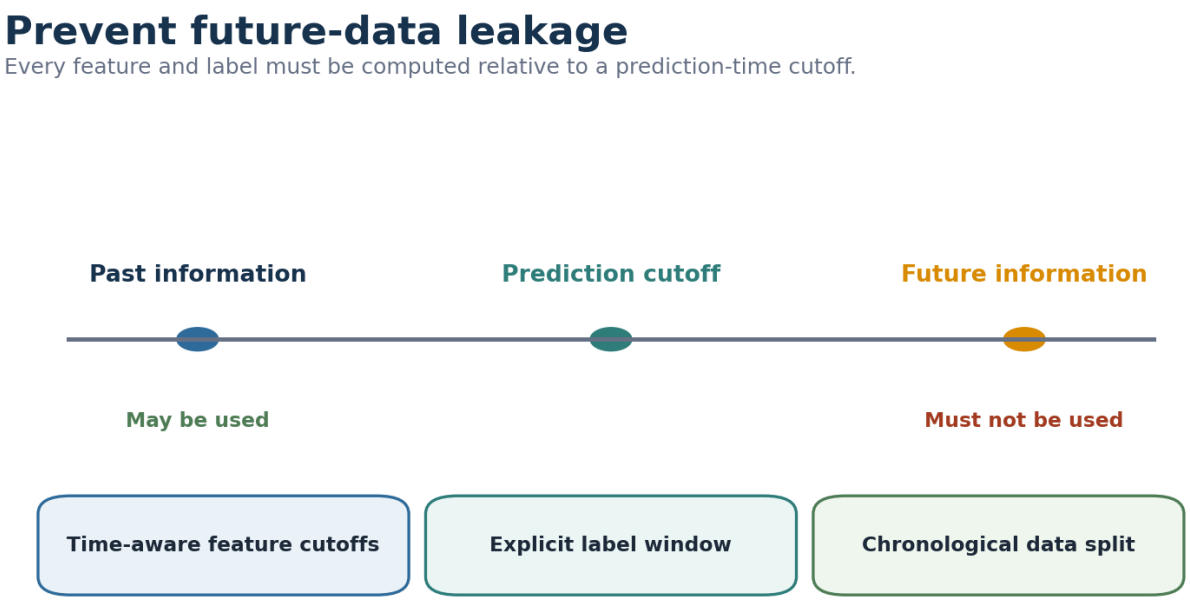



Figure 2.11 - Prediction cutoff and leakage controls.

| Leakage route | Example | Control |
|---|---|---|
| Aggregates beyond cutoff | Click count includes events after the impression | Point-in-time feature computation |
| Future campaign outcomes | Final conversion or budget result used as input | Restrict feature availability by timestamp |
| Post-click information | Conversion or dwell known only after scoring | Define an explicit label window |
| Random split across time | Later behavior appears in training for earlier evaluation | Use chronological train, validation, and test ranges |

OPEN DESIGN DISCUSSION

## Leakage audit

Feature timestamp: _____
Prediction cutoff: _____
Label window: _____
Train range: _____
Validation and test ranges: _____

# 11. Reference architecture

One defensible system combines policy filtering, scalable retrieval, rich final ranking, calibration, and point-in-time learning.

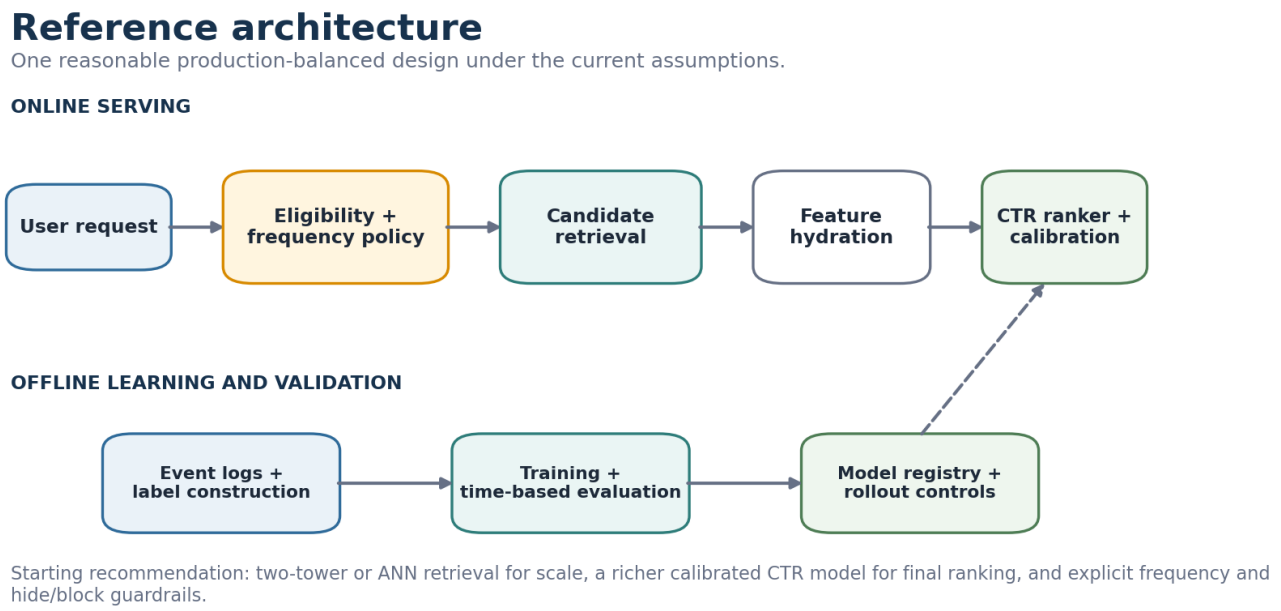



Figure 2.12 - Production-balanced reference architecture.

ONE REASONABLE DESIGN

## Current reference architecture

Apply eligibility and frequency policy first. Retrieve candidates through two-tower or ANN and other sources, merge and hydrate features, score with a richer CTR model, and calibrate probabilities. Train from point-in-time event logs, evaluate chronologically, and deploy through staged validation.

OPEN DESIGN DISCUSSION

## What would change this architecture?

Inventory-size threshold: _____
Latency threshold: _____
Cross-feature lift requirement: _____
Calibration failure condition: _____
Simpler acceptable design: _____

# 12. Decision summary

The table records a starting choice and the condition that should cause the design to change.

| Design area | Starting choice | Switch when |
|---|---|---|
| Repeated exposure | Explicit frequency or fatigue policy | Data shows no fatigue or a dynamic rule is necessary |
| Negative labels | Transparent impression-based baseline | Hidden positives materially limit quality |
| Offline metrics | Cross-entropy, normalized CE, AUC, calibration | A metric fails to predict online outcomes |
| Baseline | Logistic Regression | A still simpler prior is sufficient |
| Richer model | Compare DCN and DeepFM | Interaction lift does not justify latency or complexity |
| Two-tower placement | Candidate retrieval | Inventory is small or cross-feature loss is unacceptable |
| Final scorer | Richer calibrated CTR classifier | Two-tower similarity performs equivalently |
| Continual learning | Monitor and retrain when triggered | Stable behavior makes scheduled retraining sufficient |
| Rollout | Shadow, then limited live validation and A/B test | Risk profile requires another ordering |
| Leakage control | Point-in-time features and chronological splits | Never optional |

INTERVIEW-READY ANSWER

### Trace every component to a product or system need

I would begin with Logistic Regression, explicit label construction, normalized cross-entropy against a named baseline, and frequency guardrails. At large inventory scale, I would retrieve with a two-tower or ANN system and use a richer calibrated CTR model for final ranking. I would validate time-aware offline metrics, shadow operations, limited live traffic, A/B impact, and leakage controls before full release.

## Attribution

# Chapter 3

# People You May Know

*A temporal graph recommendation case study*

**PRIMARY REFERENCE**

## Study foundation

This chapter began as personal study notes based largely on Machine Learning System Design Interview by Ali Aminian and Alex Xu. The restructuring, clarifications, trade-off discussions, diagrams, and interpretations are the author's own.

### Chapter at a glance

| Design area | Current direction |
|---|---|
| Objective | Recommend non-connected users likely to form a meaningful future connection |
| Candidate generation | Bounded friends-of-friends plus complementary graph and cold-start sources |
| Baseline | Pointwise binary classifier or learning-to-rank model |
| Richer model | Temporal GNN for time-aware edge prediction |
| Pair scoring | Fast graph or dot-product narrowing followed by richer MLP scoring when justified |
| Serving | Precompute and cache; refresh lazily; optionally rerank with fresh lightweight features |
| Feedback | Treat repeated non-action and delayed connection attribution explicitly |

# 1. Case map and minimum scope

Generate a plausible candidate pool, estimate future connection relevance, and return a short ranked list. Do not begin with all-pairs graph prediction.

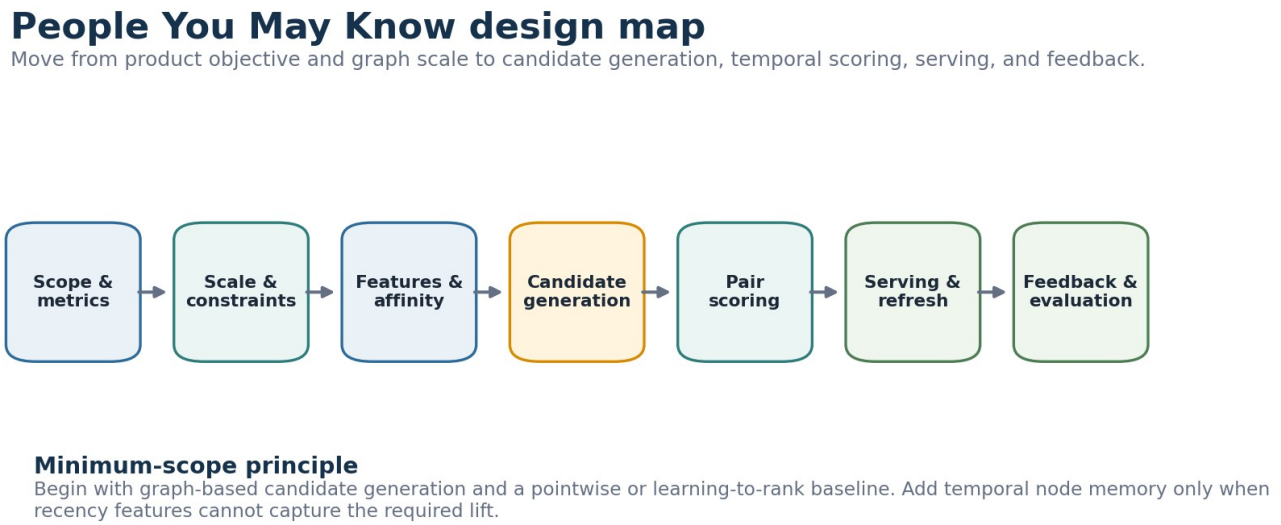



Figure 3.1 - People You May Know design map.

CONFIRMED INTERPRETATION

## Prediction internally, ranking externally

The model estimates whether a future edge is likely, while the product shows only a short ordered list. Classification and ranking metrics therefore answer different questions.

OPEN DESIGN DISCUSSION

## What is the minimum sufficient service?

Primary product outcome: _____
Candidate boundary: _____
Visible K: _____
Latency assumption: _____
Explicitly out of scope: _____

# 2. Scale assumptions and graph constraints

The notes use large illustrative numbers to reason about storage and graph expansion. They should not be presented as verified platform statistics.

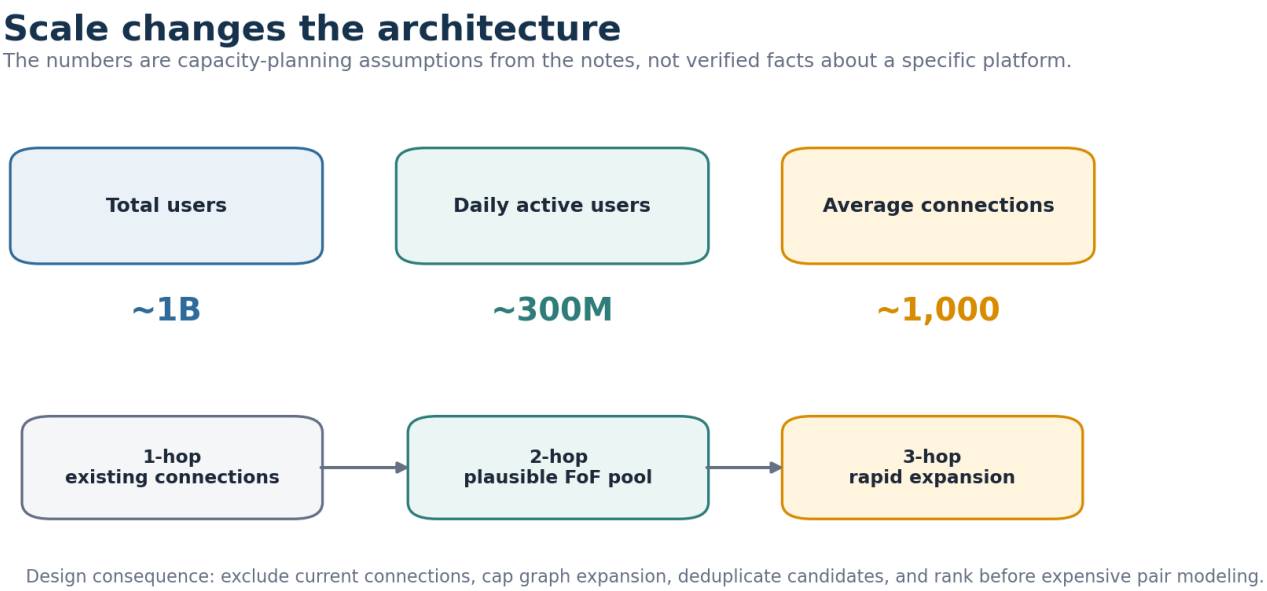



Figure 3.2 - Capacity assumptions and candidate expansion.

| Constraint | Design consequence | Reason |
|---|---|---|
| Existing 1-hop connections | Exclude from new-connection candidates | They are already connected |
| Large 2-hop pool | Cap, deduplicate, and rank candidates | Average degree can create a very large FoF set |
| 3-hop expansion | Use selectively | Coverage grows quickly with substantial cost and noise |
| Inactive users | Avoid unnecessary proactive recomputation | Not every account needs a fresh list continuously |

OPEN DESIGN DISCUSSION

### Challenge the capacity assumptions

Revised total-user assumption: _____
Revised active-user assumption: _____
Revised average degree: _____
Architecture impact: _____

# 3. Product and evaluation metrics

Requests measure recommendation activity; accepted and completed connections better represent mutual value. Offline metrics should reflect both rare positives and the visible ranked list.

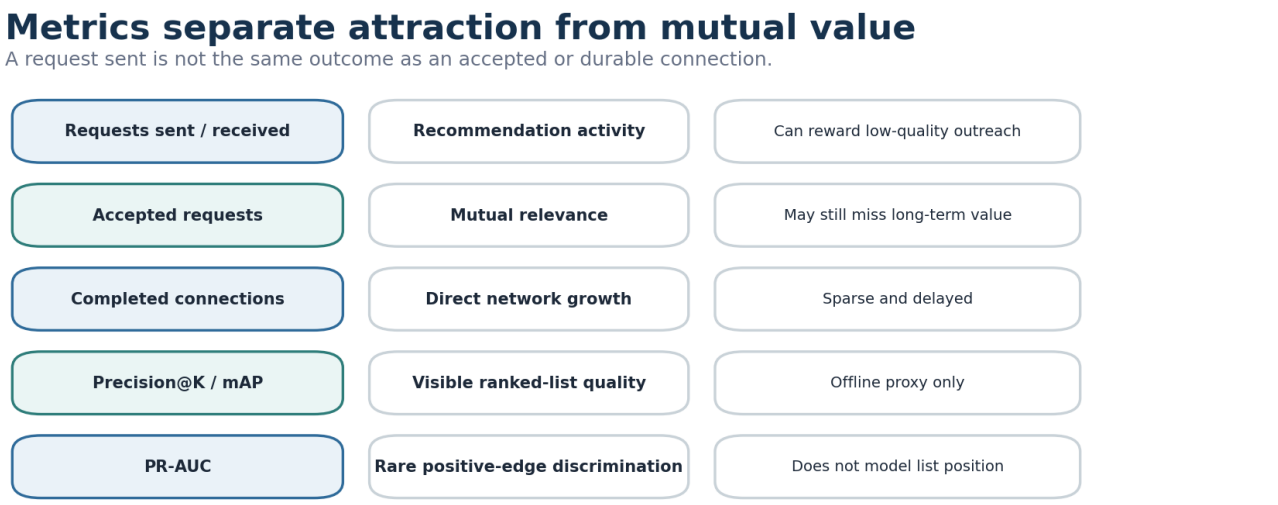



Figure 3.3 - Metric families and limitations.

| Metric family | Examples | Decision supported |
|---|---|---|
| Online activity | Requests sent and received | Do recommendations trigger outreach? |
| Mutual outcome | Accepted requests and completed connections | Did both users find the match relevant? |
| Longer-term product | Time to connection and network growth | Did the system create durable value? |
| Offline classification | PR-AUC, ROC-AUC, precision, recall | Can the model separate future edges? |
| Offline ranking | Precision@K and mAP | Is the visible list ordered well? |

**STARTING RECOMMENDATION**

### Optimize mutual outcomes, not request volume alone

Use Precision@K or mAP and PR-AUC offline, then evaluate accepted or completed connections online. Track spam, hide, block, and low-quality-request behavior as guardrails.

# 4. Baseline versus temporal graph modeling

A temporal GNN can represent evolving relationships, but it should earn its operational cost against simpler supervised and static graph baselines.

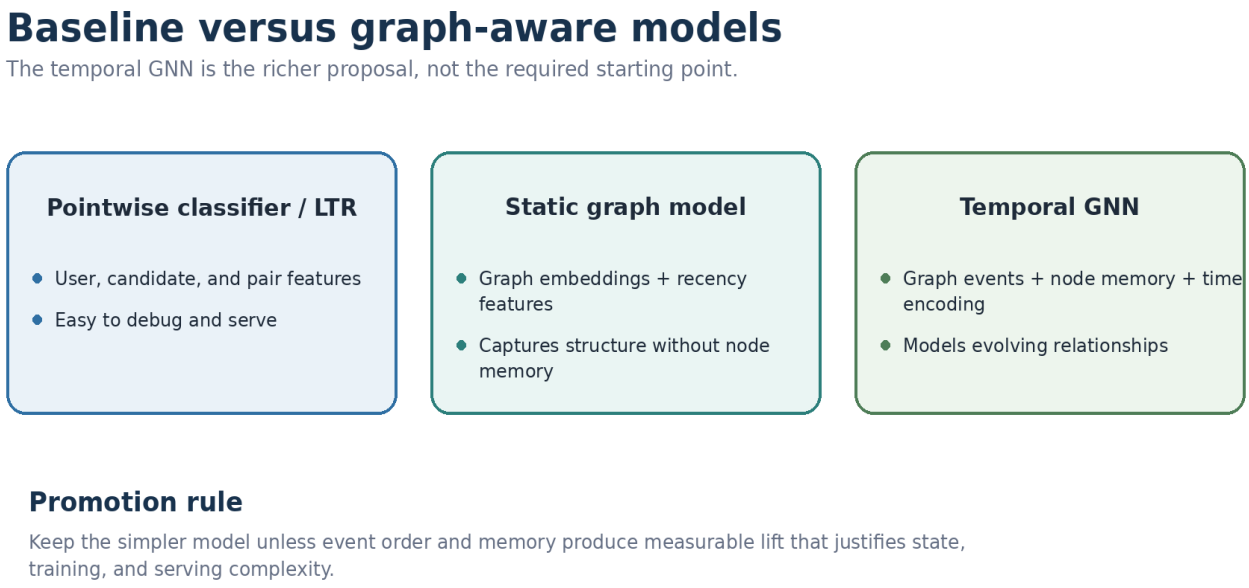



Figure 3.4 - Model alternatives by complexity.

| Model direction | Main benefit | Main cost |
|---|---|---|
| Pointwise classifier or LTR | Simple training, serving, and debugging | Requires manual graph and recency features |
| Static graph model + recency features | Captures topology without per-node temporal state | May lose event order and evolving memory |
| Temporal GNN | Models graph events, node memory, and time | Higher state, compute, and operational complexity |

OPEN DESIGN DISCUSSION

### Is temporal memory justified?

Expected lift over pointwise baseline: _____
Event order that matters: _____
Static-model alternative: _____
Complexity threshold for rejection: _____

# 5. Data, features, and social affinity

Represent users, graph structure, direct activity, recency, and exposure history. Standardize education and work text before treating it as categorical affinity.

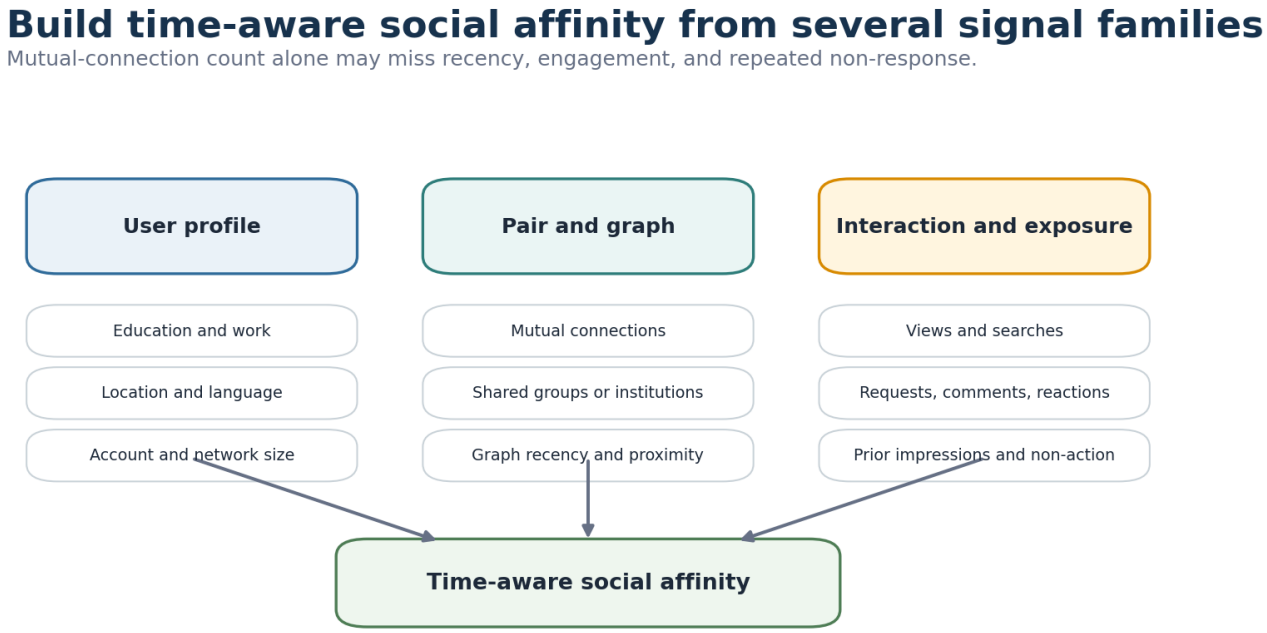



Figure 3.5 - Feature families that form time-aware social affinity.

| Feature family | Examples | Important control |
|---|---|---|
| User profile | School, degree, major, work, location, language | Normalize equivalent text such as CS and Computer Science |
| Graph | Mutual connections, graph hops, shared institutions | Use only information available before scoring |
| Direct interaction | Profile views, searches, requests, comments, reactions | Preserve timestamp and direction |
| Exposure | Prior recommendation impressions and time since exposure | Distinguish never shown from repeatedly ignored |

CONFIRMED INTERPRETATION

### Recency alone is insufficient

Social affinity may combine recent mutual connections, direct engagement, sustained older engagement, graph changes, and repeated opportunities without action.

# 6. Temporal GNN architecture and node memory

At each event time, build messages from users, edges, features, and time; update memory; create embeddings; and score a future pair.

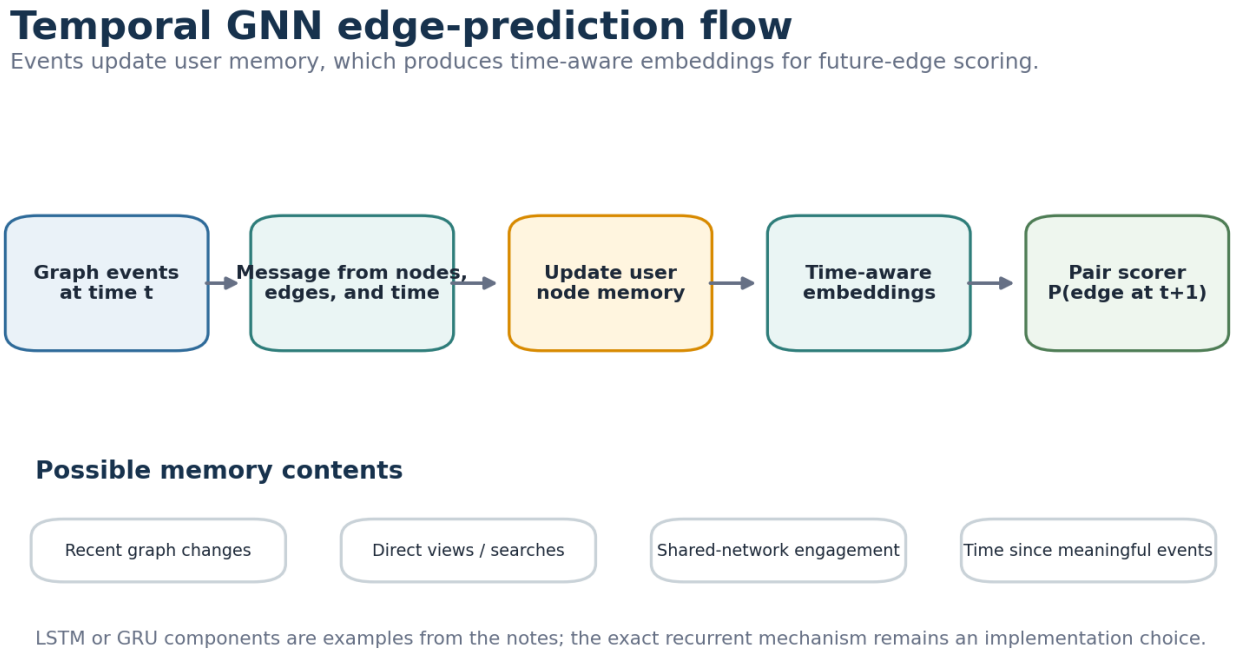



Figure 3.6 - Temporal GNN edge-prediction flow.

| Memory design question | Possible content | Trade-off |
|---|---|---|
| Event coverage | Connections, views, searches, requests, shared-network engagement | More event types increase complexity and noise |
| History length | Fixed window or time decay | Longer history costs more and may preserve stale behavior |
| Per-user memory | Compact recurrent state | Larger state may improve detail but harms serving cost |
| Inactive users | Decay, archive, or lazy reactivation | Aggressive cleanup may lose useful history |

**OPEN DESIGN DISCUSSION**

### What should node memory retain?

Event types: _____
History or decay rule: _____
Per-user state budget: _____
Inactive-user handling: _____

# 7. Candidate generation and graph alternatives

All-pairs prediction is infeasible. Candidate generation should use graph proximity and complementary sources before rich edge prediction.

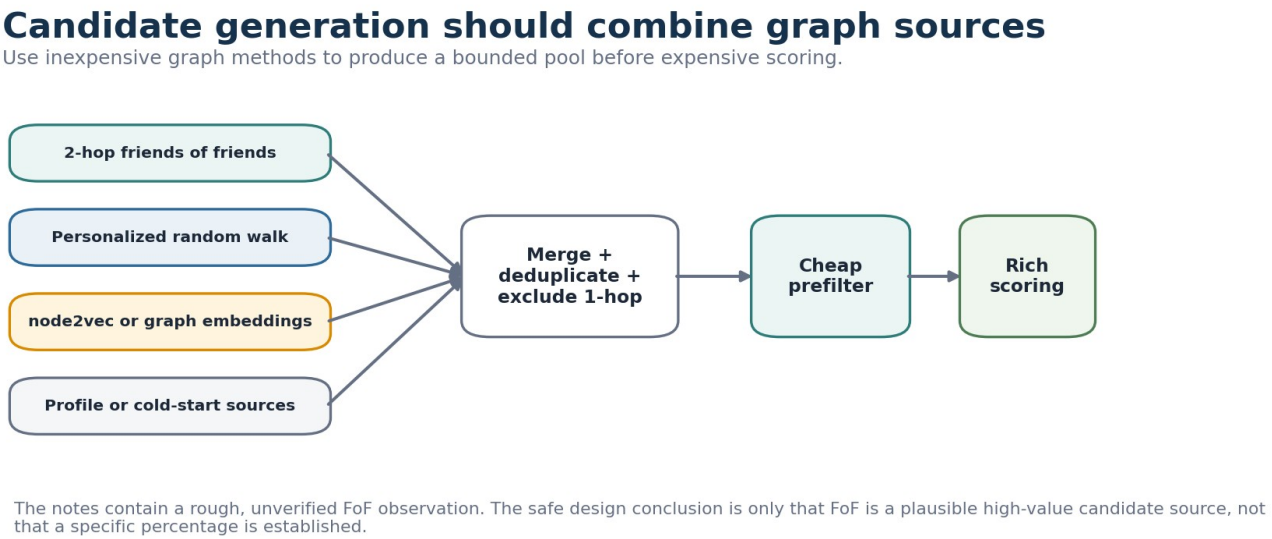



Figure 3.7 - Candidate-source merge and staged scoring.

| Candidate source | Best when | Main limitation |
|---|---|---|
| 2-hop friends of friends | Graph structure is informative and mutual connections matter | May under-cover sparse or new users |
| Personalized random walk | Local graph proximity beyond raw hop count is useful | Can be costly or biased toward dense regions |
| node2vec / graph embedding | Reusable graph representations support retrieval | May become stale and lose event timing |
| Profile or contextual source | Cold-start users lack graph history | May be weaker than real social signals |

SOURCE CAUTION

### Do not publish the rough FoF percentage as verified fact

The original notes record an approximate external observation. This public chapter keeps only the design implication that friends-of-friends are a plausible high-value source.

# 8. Pair scoring: dot product, MLP, or both

Fast similarity scoring supports large candidate pools; richer pair modeling supports final ranking. They may be alternatives or sequential stages.

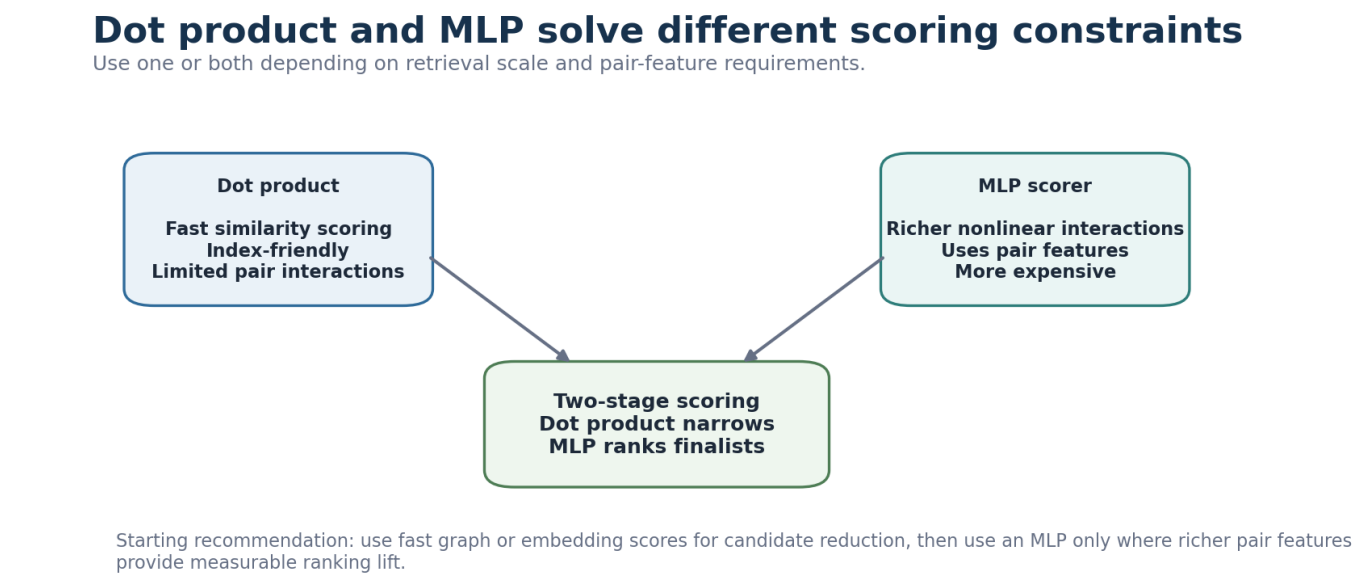



Figure 3.8 - Pair-scoring alternatives.

| Scorer | Best when | Main limitation |
|---|---|---|
| Dot product | Embedding retrieval and high-throughput narrowing | Limited interaction flexibility |
| MLP | Pair features and nonlinear interactions matter | More expensive and less index-friendly |
| Dot product then MLP | Scale and rich final scoring are both required | Two-stage pipeline and score alignment |

STARTING RECOMMENDATION

### Use rich scoring only on a bounded set

Generate and narrow candidates with graph rules, embeddings, or a lightweight scorer. Apply an MLP only to the smaller finalist set when pair-feature lift justifies the cost.

# 9. Serving, precomputation, and lazy refresh

The social graph changes more slowly than a content feed, so cached recommendations can reduce cost without recomputing after every event.

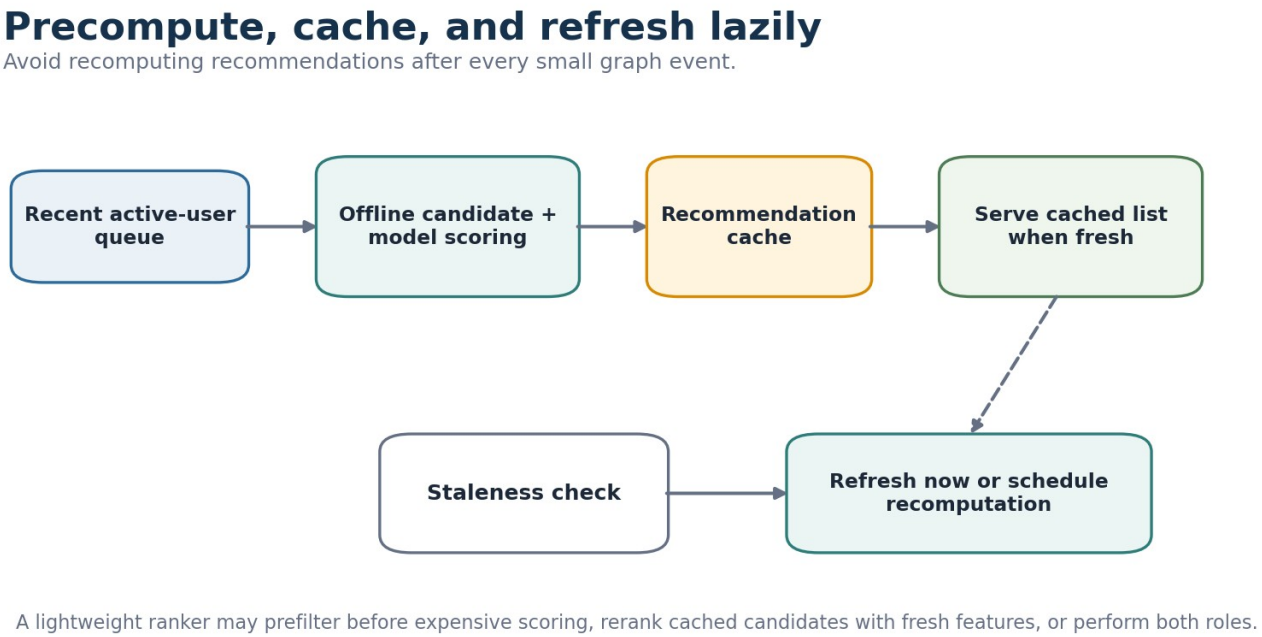



Figure 3.9 - Cached serving with lazy refresh.

| Serving choice | Benefit | Risk |
|---|---|---|
| Precompute for recently active users | Focuses compute on likely traffic | A returning inactive user may receive stale results |
| Lazy refresh on request | Avoids event-by-event recomputation | First stale request may need fallback behavior |
| Lightweight fresh reranker | Uses recent features without full recomputation | Adds online feature and model dependencies |
| Immediate graph-event refresh | Maximum freshness | Often unnecessary and expensive |

CONFIRMED INTERPRETATION

### Precompute plus lazy refresh is the preferred serving idea

Serve the cached list when it is fresh. When stale, refresh immediately or schedule recomputation. A lightweight ranker may prefilter, rerank cached candidates, or do both.

# 10. Missing profiles, exposure, and delayed feedback

The service should distinguish unavailable profile fields from actual values and should treat repeated non-action differently from a single ignored impression.

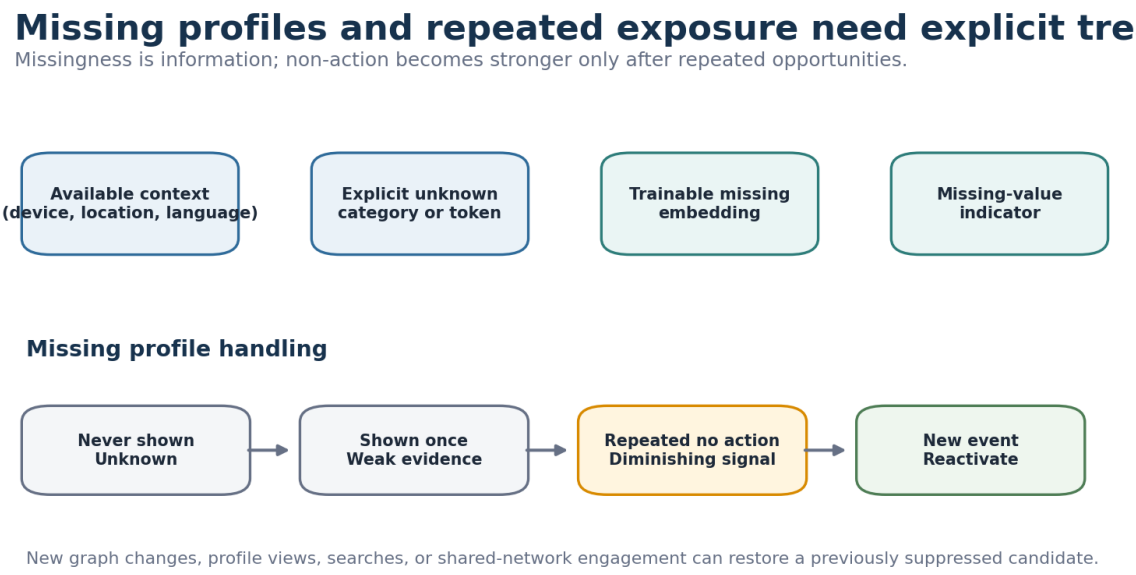



Figure 3.10 - Missing-value options and exposure states.

| Problem | Options | Decision principle |
|---|---|---|
| Missing profile fields | Available context, unknown token, trainable embedding, indicator | The options may be combined; model why the value is missing |
| Ignored recommendation | No penalty, decay, temporary suppression | One ignore is weak; repeated exposure is stronger evidence |
| Candidate restoration | New graph or engagement signals | Do not suppress permanently when the relationship changes |
| Delayed connection credit | Fixed window, decay, recent-touch, multi-touch, dynamic attribution | Use time and intervening events, not elapsed time alone |

**OPEN DESIGN DISCUSSION**

## How should ignored and delayed outcomes be attributed?

Exposure penalty: _____
Suppression duration: _____
Reactivation signal: _____
Label window: _____
Attribution method: _____

# 11. Negative-pair sampling and online evaluation

Negative-pair sampling is an added technical extension because future connections are rare and naive random negatives may be too easy.

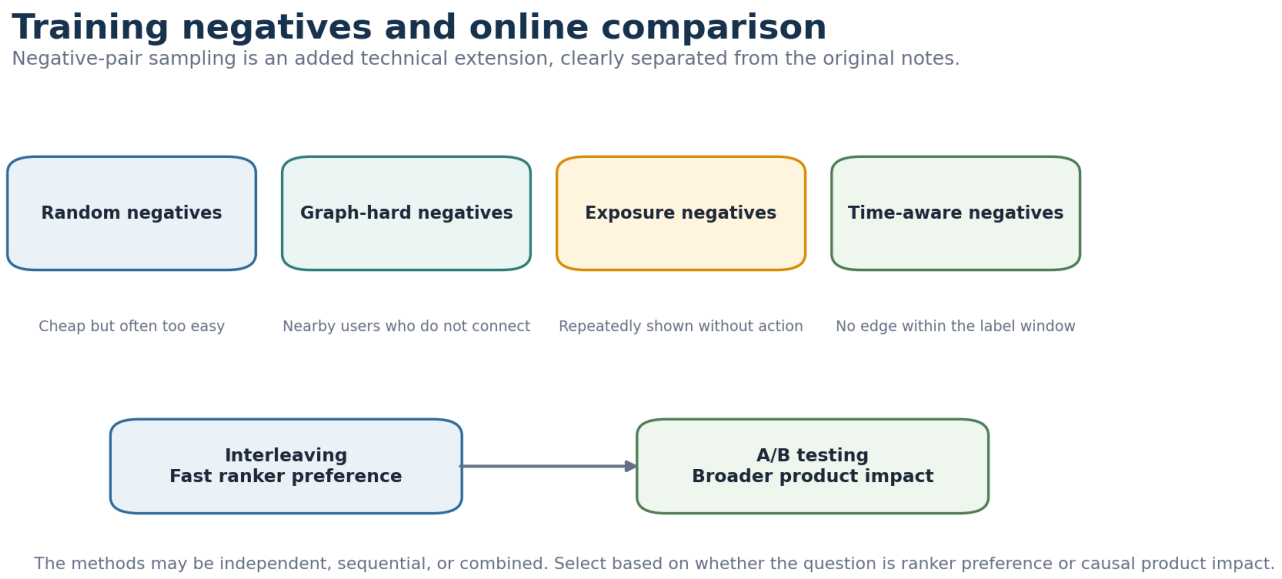



Figure 3.11 - Negative-pair and online-evaluation options.

| Negative strategy | Benefit | Risk |
|---|---|---|
| Random users | Simple and scalable | Often trivially easy and unlike serving candidates |
| Graph-hard negatives | Matches plausible nearby users | May include future positives |
| Exposure negatives | Reflects repeated opportunity without action | Affected by ranking and exposure bias |
| Time-aware negatives | Respects a defined future-edge window | Window choice changes the label |

| Online method | Best use | Limitation |
|---|---|---|
| Interleaving | Fast preference comparison between rankers | Narrower than full product impact |
| A/B testing | Causal effect on connections and guardrails | Slower and requires traffic |
| Sequential or combined | Fast ranking signal followed by product validation | More experiment coordination |

# 12. Reference architecture

One defensible system combines bounded graph candidates, supervised scoring, cache-aware serving, explicit exposure policy, and evidence-based temporal-model promotion.

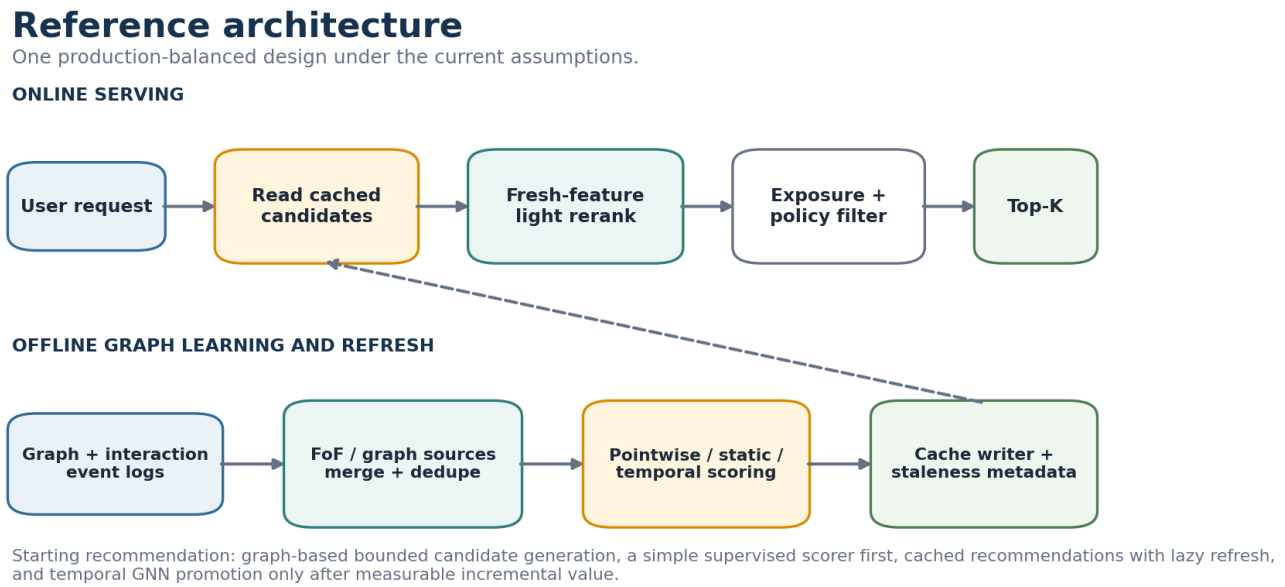



Figure 3.12 - Production-balanced reference architecture.

ONE REASONABLE DESIGN

## Current reference architecture

Build candidates from bounded friends-of-friends and complementary graph or profile sources, merge and exclude current connections, score with a pointwise or static model first, write a cached list with staleness metadata, and rerank lightly at request time. Promote a temporal GNN only when event memory produces meaningful lift.

OPEN DESIGN DISCUSSION

## What would change this architecture?

Candidate-pool threshold: _____
Freshness requirement: _____
Temporal-model lift requirement: _____
Per-user state budget: _____
Simpler acceptable design: _____

# 13. Decision summary

The starting choices remain deliberately reversible. Each switch condition identifies the evidence required to add or remove complexity.

| Design area | Starting choice | Switch when |
|---|---|---|
| Product outcome | Accepted or completed connections | Request activity is proven to be the right business target |
| Candidate source | Bounded 2-hop FoF plus complementary sources | Coverage or quality requires broader graph traversal |
| Baseline | Pointwise classifier or LTR | A static graph model clearly improves quality |
| Richer model | Temporal GNN as a promoted experiment | Temporal lift does not justify state and serving cost |
| Affinity | Graph, recency, engagement, and exposure features | A smaller feature set performs equivalently |
| Pair scoring | Cheap narrowing then MLP finalists | One scorer satisfies both scale and quality |
| Serving | Precompute, cache, and lazy refresh | Freshness loss harms outcomes |
| Missing data | Explicit missingness plus available context | A simpler treatment performs equivalently |
| Ignored items | Gradual penalty and temporary suppression | Non-action is shown to be non-informative |
| Evaluation | Top-K + rare-positive metrics, then online test | Offline metrics fail to predict product impact |

INTERVIEW-READY ANSWER

### Trace every component to scale, freshness, or relationship quality

I would begin with bounded friends-of-friends and complementary candidate sources, then use a pointwise or learning-to-rank baseline with graph, profile, recency, and exposure features. I would cache lists and refresh lazily, use a lightweight fresh reranker when needed, and promote a temporal GNN only after measurable lift over static recency features. I would evaluate Precision@K or mAP and PR-AUC offline, then accepted or completed connections with safety guardrails online.

## Attribution

This project is independent study work and is not affiliated with the book authors or publisher. It is not a substitute for the original book. Public versions use original wording and original diagrams and exclude handwritten source scans.

# HARMFUL CONTENT DETECTION

A visual system-design case study

**MINIMUM SUFFICIENT DESIGN**

Start with a credible baseline. Add granularity or complexity only when a requirement or measured limitation justifies it.

| | |
|---|---|
| PRIMARY SCOPE<br>**Post-level multimodal moderation** | PREFERRED MODEL<br>**Early-fusion shared-bottom multi-task model** |
| SERVICE ACTION<br>**Keep · demote · review · remove** | MAIN BASELINE<br>**Shared multi-label classifier** |
| LABEL STRATEGY<br>**Weak reports + trusted human labels** | ROLLOUT<br>**Shadow deployment first** |

**Hsiang-Yu Hsieh**

ML System Design Handbook · Chapter 4

# 0 How to read this chapter

The chapter separates source inspiration, confirmed interpretation, open decisions, and starting recommendations.

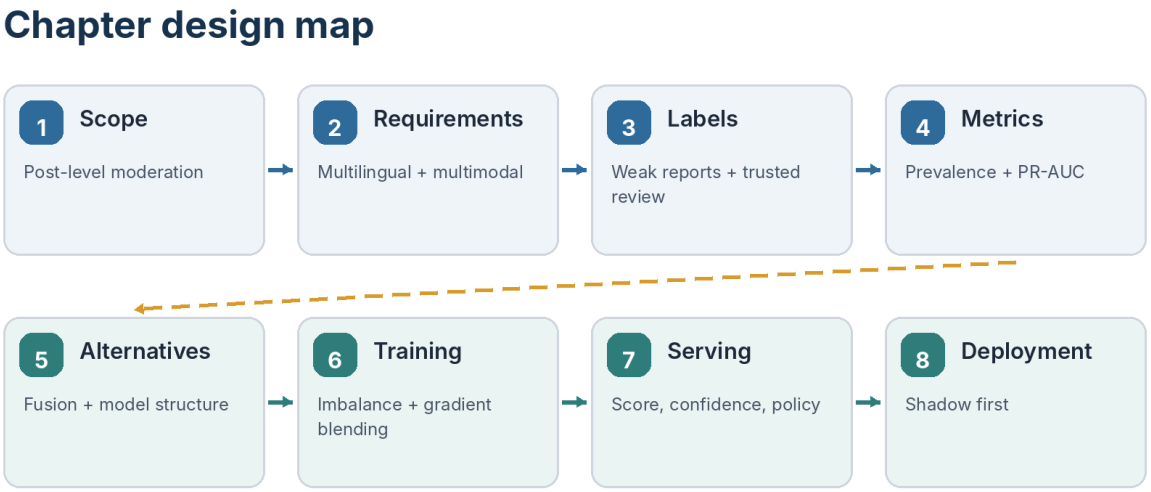



*Figure 0.1 - The recurring order used in this case.*

FROM THE ORIGINAL NOTES

## Source-derived content

Source callouts preserve the concepts and options present in the original study notes.

CONFIRMED INTERPRETATION

## Clarified meaning

Teal callouts record the decisions confirmed during discussion.

OPEN DESIGN DISCUSSION

## Reader decision

Amber boxes preserve genuine alternatives and provide a compact place to choose and defend an option.

STARTING RECOMMENDATION

## One defensible design

Green callouts show a reasonable starting point under the current assumptions, not a universal answer.

# 1 Scope and service decision

Define the main service before selecting a model.

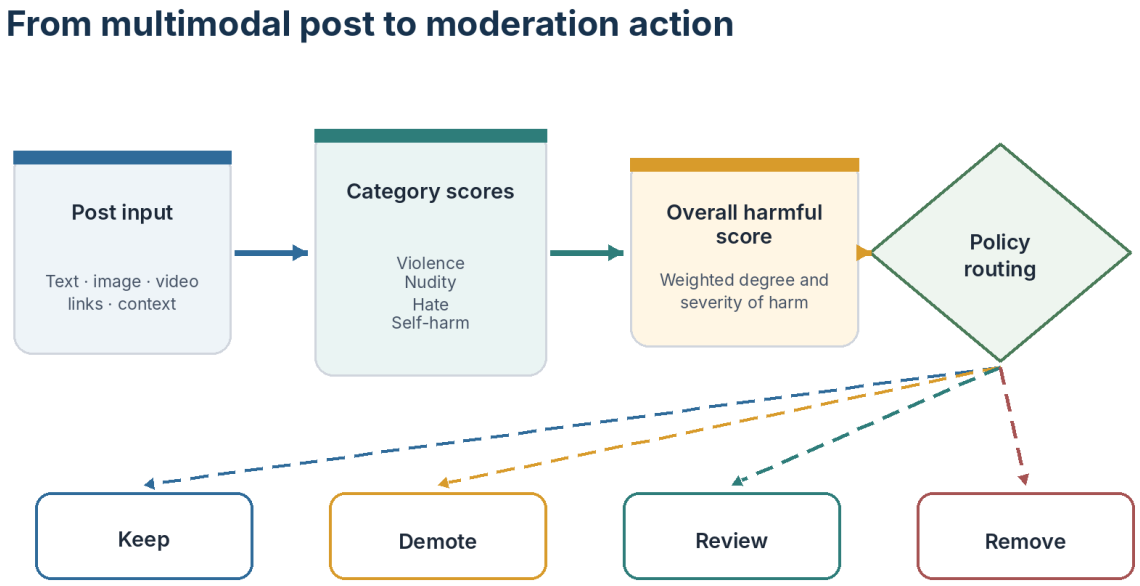



*Figure 1.1 - Granular model outputs support a policy-level moderation action.*

CONFIRMED INTERPRETATION

## Multi-label internally, policy-driven externally

The model produces several category-level harm scores. The service aggregates them into an overall harmful score and routes the post to keep, demote, manual review, or remove.

| In scope | Related but separate | Out of scope |
|---|---|---|
| **Post-level harmful content** | Bad-actor risk across behavior over time | User-facing explanation generation |
| **Text, image, and video** | Misinformation as a future case | Full actor-level architecture |
| **Category scores and moderation action** | Account-level enforcement signals | Detailed misinformation policy |

DESIGN DECISION

**What exactly must this service decide?**

**Primary unit of prediction:** _____
**Granular model outputs:** _____
**Final product action:** _____
**Related system boundary:** _____

# 2 Requirements and latency

The architecture should follow the severity, modality, and operating constraints.

| Requirement | Design consequence | Why it matters |
|---|---|---|
| **Multilingual text** | Use multilingual text representations | Harmful language varies across regions and languages. |
| **Text + image + video** | Use modality encoders and a fusion strategy | Meaning may depend on more than one modality. |
| **Violence may be urgent** | Maintain a real-time path | Some severe content cannot wait for a batch job. |
| **Other categories may tolerate delay** | Allow asynchronous or batch scoring | Not every category needs the most expensive path. |
| **Policy action requires confidence** | Return score and confidence | Borderline cases may need review rather than removal. |

STARTING RECOMMENDATION

## Split latency by severity

Use real-time inference for severe or urgent categories and asynchronous rescoring for lower-urgency categories or signals that arrive later, such as comments.

DESIGN DECISION

**Which requirements actually change the architecture?**

**Real-time categories:** _____

**Batch-eligible categories:** _____

**Maximum online latency:** _____

**Required modalities:** _____

# 3 Labels and evaluation

Use reports for scale and human review for trust.

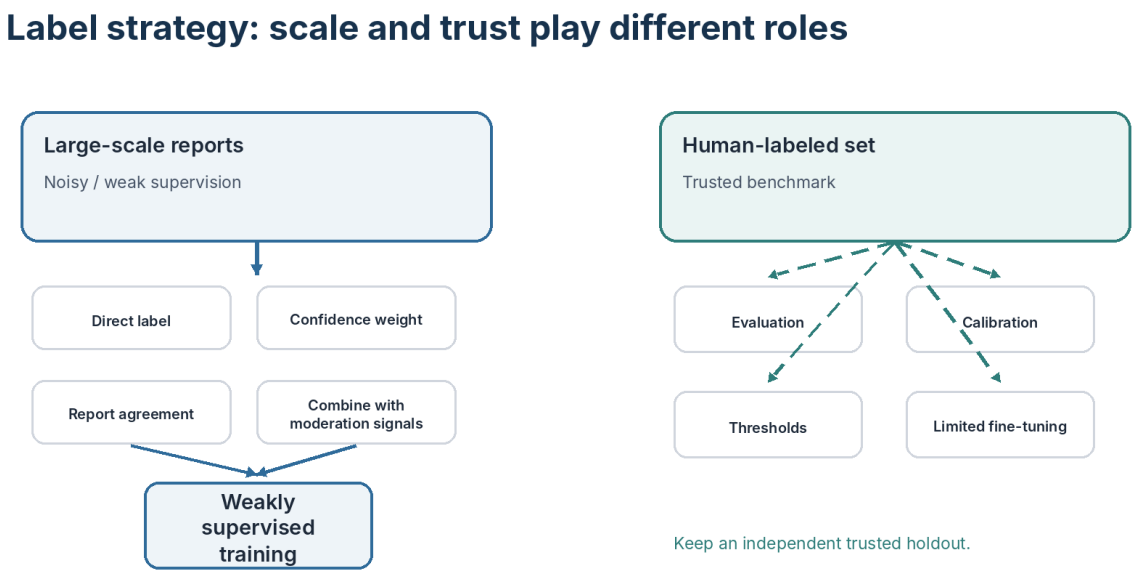



*Figure 3.1 - Weak supervision and a trusted benchmark serve different purposes.*

FROM THE ORIGINAL NOTES

## User reports are the scalable label source

Reports reduce annotation cost, but they may be incomplete, incorrect, or malicious.

CONFIRMED INTERPRETATION

## Human labels remain the trusted benchmark

Reserve carefully reviewed examples for evaluation, calibration, and threshold selection. Limited fine-tuning is possible only after keeping an independent holdout.

DESIGN DECISION

**How should a user report become a training label?**

**Single-report treatment:** _____
**Multiple-report agreement:** _____
**Filtering rule:** _____
**Trusted holdout size:** _____

### Product and offline metrics

| Metric | What it measures | Best use |
|---|---|---|
| **Content prevalence** | Harmful posts that remain visible / total posts | Volume of harmful content |
| **Exposure prevalence** | Harmful impressions / total impressions | Actual user exposure |

| Metric | What it measures | Best use |
|---|---|---|
| **Valid appeal rate** | Reversed decisions / appealed decisions | Moderation quality |
| **Proactive detection** | Confirmed harm found before reports / all confirmed harm | System initiative |
| **PR-AUC** | Positive-class precision-recall trade-off | Rare harmful categories |
| **ROC-AUC** | Broad score discrimination | Secondary diagnostic under imbalance |

STARTING RECOMMENDATION

### Choose one product metric and several guardrails

Exposure prevalence often matches user harm more directly than raw post counts. Pair it with appeal rate, proactive detection, and category-level quality.

# 4 Alternative 1: multimodal fusion

Compare the modular baseline with the cross-modal proposal.

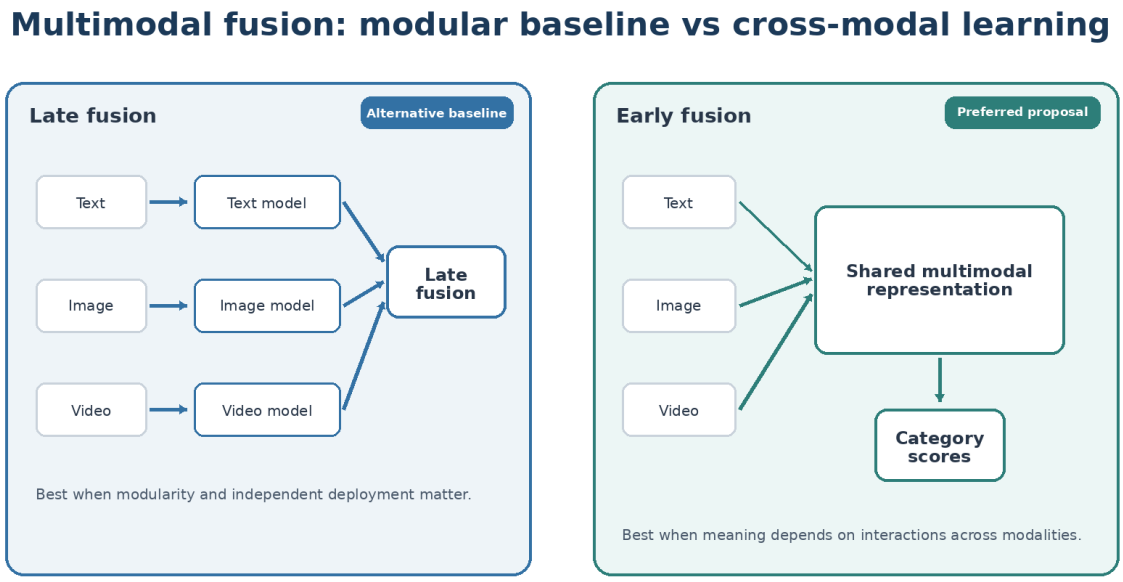



*Figure 4.1 - Late fusion is operationally simple; early fusion can learn cross-modal meaning.*

| Option | Best when | Main benefit | Main cost |
|---|---|---|---|
| **Late fusion** | Modularity and independent deployment matter | Easier training, debugging, and missing-modality handling | May combine information too late |
| **Early fusion** | Meaning depends on modality interaction | Learns cross-modal relationships in one shared model | Harder training and higher compute |

STARTING RECOMMENDATION

## Use late fusion as the implementation baseline and early fusion as the preferred richer model

Move to early fusion when cross-modal interactions produce measurable lift over the modular baseline.

DESIGN DECISION

**What evidence justifies early fusion?**

**Cross-modal failure examples:** _____

**Quality lift over late fusion:** _____

**Missing-modality behavior:** _____

**Latency and cost budget:** _____

# 5 Alternative 2: prediction architecture

Increase output granularity only when the service needs it.

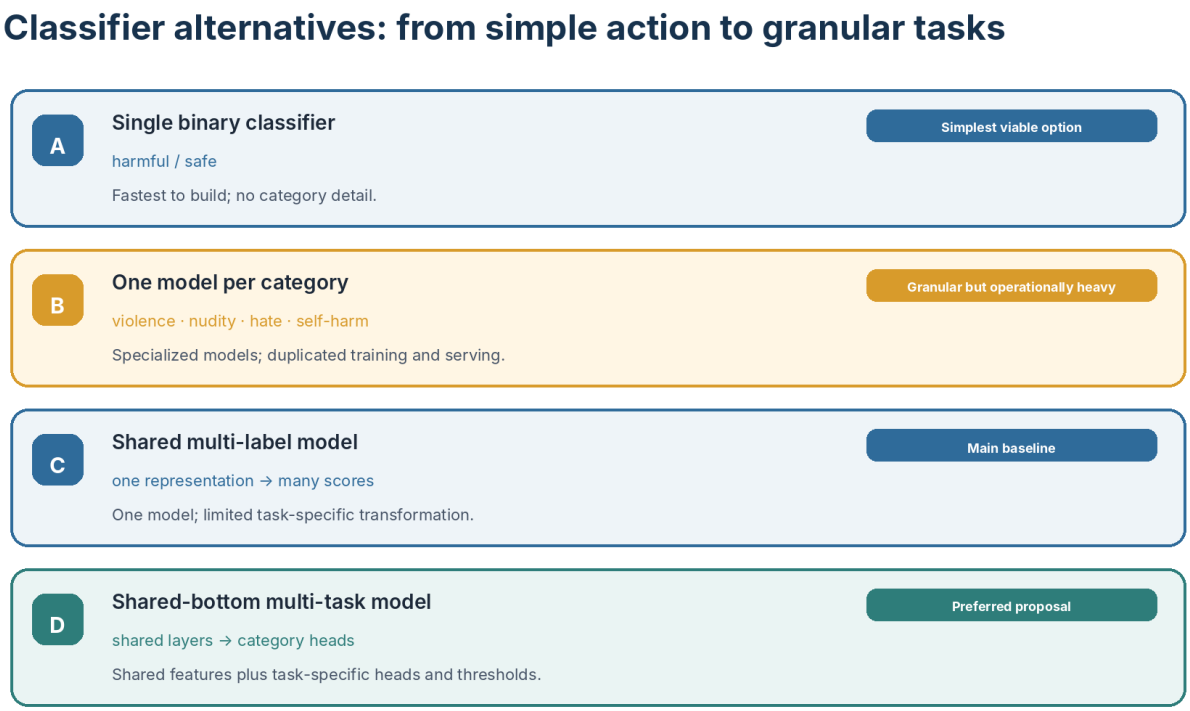



*Figure 5.1 - The alternatives move from one binary action to shared, category-specific learning.*

**EASY TO IMPLEMENT** **Single binary classifier**

| BEST WHEN | MAIN BENEFIT |
|---|---|
| The service only needs harmful / safe and category detail is unnecessary. | Fastest implementation and simplest serving. |

**MAIN COST** No category-level analysis or severity weighting.

**GRANULAR BUT HEAVY** **One classifier per category**

| BEST WHEN | MAIN BENEFIT |
|---|---|
| Each category needs a highly specialized model and separate operations are acceptable. | Strong category specialization. |

**MAIN COST** Duplicated training, deployment, monitoring, and inference.

**MAIN BASELINE** **Shared multi-label classifier**

| BEST WHEN | MAIN BENEFIT |
|---|---|
| One model should predict several independent categories with minimal complexity. | Shared representation and a single serving unit. |

**MAIN COST** Limited task-specific transformation.

| PREFERRED PROPOSAL **Shared-bottom multi-task model** | PRODUCTION CANDIDATE |
|---|---|
| **BEST WHEN**<br>Categories share useful features but require different transformations, losses, and thresholds. | **MAIN BENEFIT**<br>Balances shared learning and category specialization. |
| **MAIN COST** More complex loss weighting and debugging. | |

CONFIRMED INTERPRETATION

### The service still ends in one moderation policy

Granular heads support analysis and severity weighting. They do not require the product to expose multiple decisions to the user.

# 6 Representations and features

Keep model examples specific enough for implementation, but not mandatory.

| Input | Illustrative encoder from the notes | Role |
|---|---|---|
| **Text** | DistilBERT, Sentence-BERT, multilingual embeddings | Language representation |
| **Image** | CLIP visual encoder or SimCLR | Visual representation |
| **Video** | VideoMoCo | Temporal visual representation |
| **Context** | Feature preparation | Posting time, device, location, selected account signals |

CONFIRMED INTERPRETATION

### Encoder names are examples, not requirements

The purpose of the examples is to show the level of implementation specificity possible in an interview or internal design, while leaving room for actual benchmarking.

## Missing modalities

| Alternative | Benefit | Risk |
|---|---|---|
| **Zero vector** | Simplest representation | May resemble a valid low-information embedding |
| **Special token or vector** | Makes missingness explicit | Adds a learned or engineered representation |
| **Encoder-specific handling** | Fits each modality | Less uniform system behavior |

## Comments and author history

| Feature | Simple default | Open question |
|---|---|---|
| **Comments** | Average a bounded set of comment embeddings | Use only for later rescoring, or move to attention / top-k / thread-aware aggregation |
| **Author history** | Use only information available before the current post | Include in harmful score, use for review routing, or exclude to reduce bias |

DESIGN DECISION

**Which features belong in the main score versus later routing?**

**Initial moderation features:** _____
**Later rescoring features:** _____
**Author-history role:** _____
**Leakage guardrail:** _____

# 7 Overall harmful score and reference architecture

One reasonable design under the current assumptions.

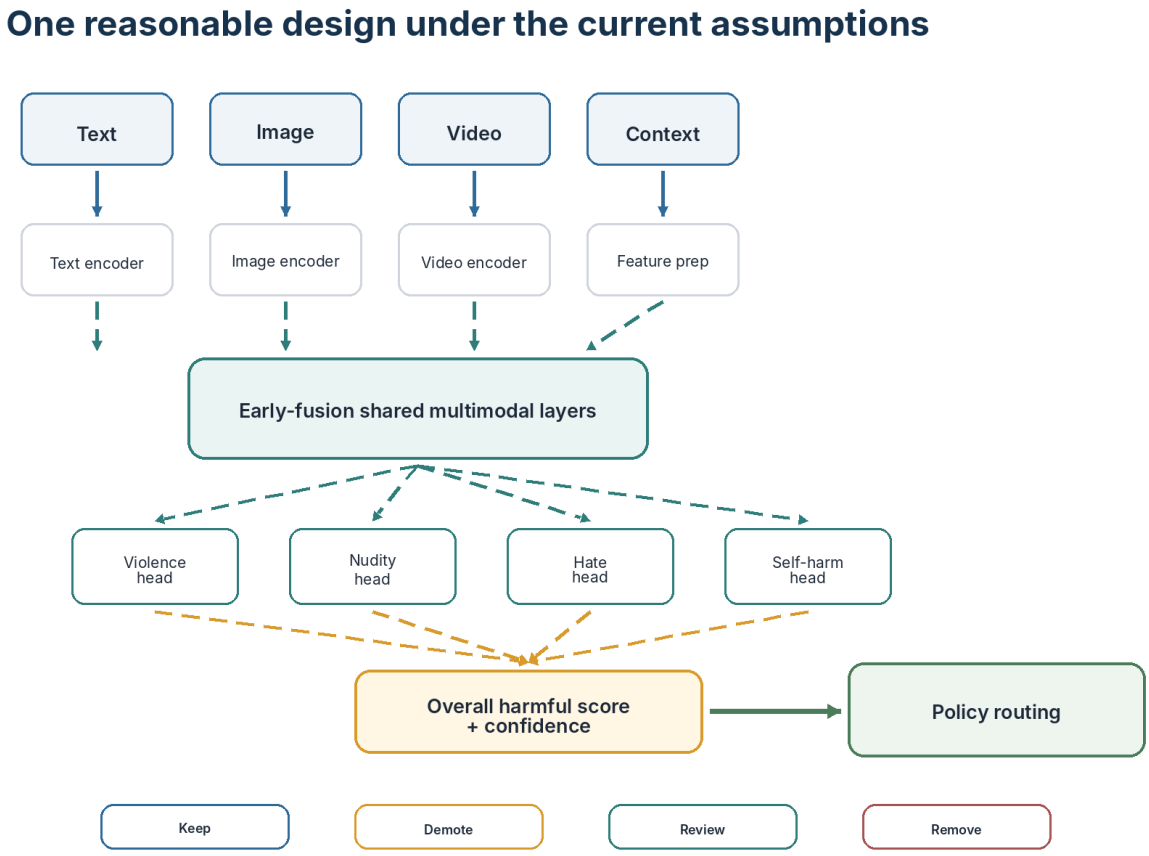



*Figure 7.1 - Early fusion, category heads, score aggregation, and policy routing.*

| Aggregation option | Benefit | Main limitation |
|---|---|---|
| **Weighted category combination** | Transparent severity weighting | Weights require policy and validation decisions |
| **Highest-risk category** | Simple and sensitive to one severe category | Ignores combinations of moderate scores |
| **Learned aggregation** | Models interactions among categories | Less transparent and needs suitable labels |
| **Hybrid** | Combines learned score with severe-category rules | More policy and maintenance complexity |

STARTING RECOMMENDATION

## Use a hybrid aggregation layer

Use the learned or weighted score for general cases and explicit overrides for severe categories when policy requires them.

DESIGN DECISION

**How should category scores become one harmful score?**

**Aggregation method:** _____
**Severe-category override:** _____
**Multiple moderate scores:** _____
**Calibration plan:** _____

# 8 Training and class imbalance

Use only the methods needed by observed failure modes.

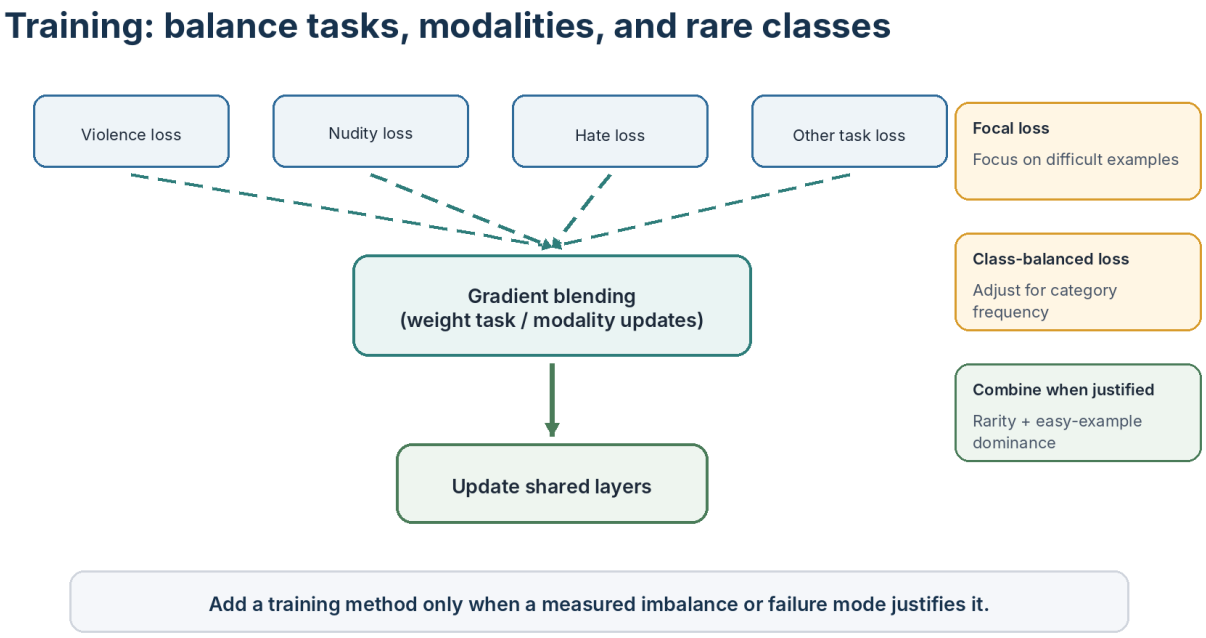



*Figure 8.1 - Task losses feed a blended shared-layer update; imbalance methods solve different problems.*

| Method | Use when | Benefit | Risk |
|---|---|---|---|
| **Gradient blending** | One task or modality dominates shared updates | Balances influence on shared layers | Weights add tuning and debugging complexity |
| **Focal loss** | Easy safe examples dominate | Focuses learning on difficult examples | May emphasize noisy reports |
| **Class-balanced loss** | Category counts are highly unequal | Gives rare categories more influence | May overweight noisy rare classes |
| **Combined loss** | Both rarity and easy-example dominance are measured | Addresses two imbalance sources | More hyperparameters and interaction effects |

MINIMAL-SCOPE RULE

## Do not add every optimization method

Start with the simplest loss. Add gradient blending, focal loss, or class-balanced weighting only after measuring the corresponding imbalance.

DESIGN DECISION

**Which training method is justified by the data?**

**Observed imbalance:** _____
**Selected method:** _____
**Noise concern:** _____
**Evidence of improvement:** _____

# 9 Serving, latency, and deployment

Route by risk and validate silently before enforcement.

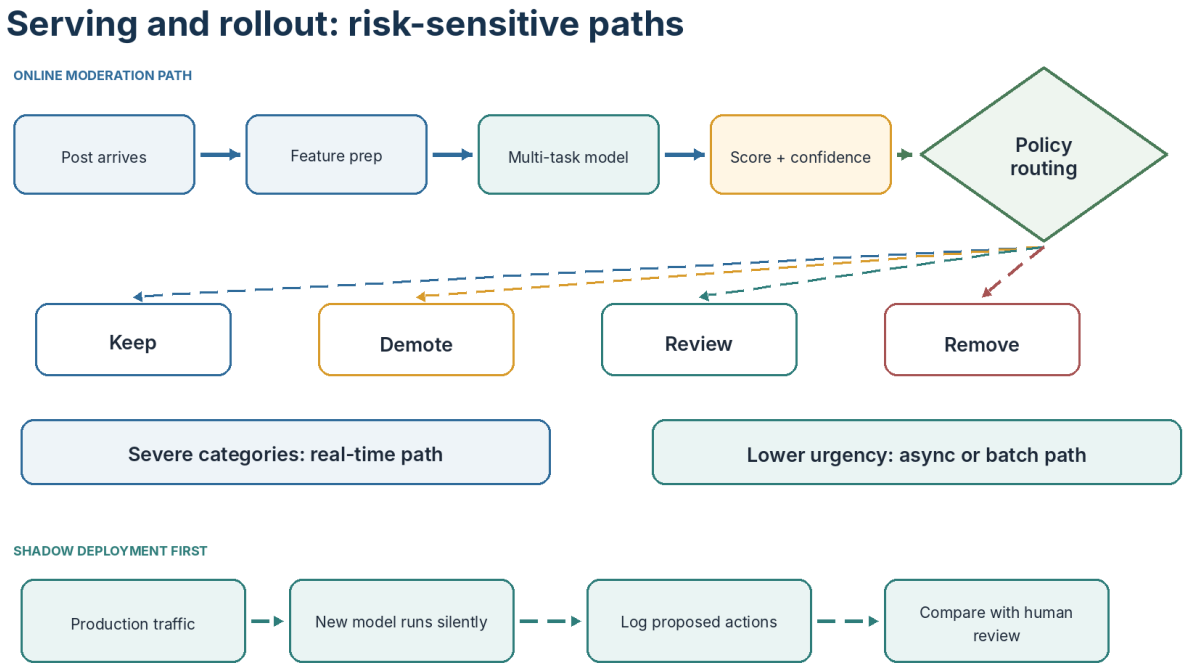



*Figure 9.1 - The online moderation path and the safer first rollout step.*

| Score / confidence region | Action | Reason |
|---|---|---|
| **Low risk** | Keep | Avoid unnecessary intervention |
| **Moderate risk** | Demote or limit distribution | Reduce exposure without immediate removal |
| **Borderline or uncertain** | Manual review | Use human judgment where error cost is high |
| **High risk** | Remove and possibly record a violation | Act quickly on severe, confident harm |

STARTING RECOMMENDATION

## Shadow deployment before user-facing enforcement

Run the new model on production traffic, log its proposed actions, and compare with the current system and trusted human review.

| Shadow deployment can validate | It cannot fully validate |
|---|---|
| **Latency, reliability, score distribution, disagreement cases, human-review agreement** | User behavior after real demotion or removal |

DESIGN DECISION

**What evidence permits limited enforcement?**

**Shadow metrics:** _____
**Human-review agreement:** _____
**High-risk disagreement review:** _____
**Rollback condition:** _____

# 10 Decision summary

A concise interview-ready reference design.

ONE REASONABLE DESIGN

## Current reference architecture

Use an early-fusion shared-bottom multi-task model, train at scale with weak report labels and evaluate on trusted human labels, aggregate category scores into a calibrated harmful score, and route content by score, confidence, category severity, and latency requirement.

| Design area | Starting choice | Switch when |
|---|---|---|
| **Fusion** | Early fusion after a late-fusion baseline | Cross-modal lift is not measurable or operations dominate |
| **Model structure** | Shared-bottom multi-task | Shared multi-label performs similarly with much lower complexity |
| **Labels** | Reports for scale; human labels for trust | Report quality is too weak for useful supervision |
| **Aggregation** | Hybrid score + severe-category overrides | Policy can be represented transparently by one simpler rule |
| **Latency** | Real-time severe path; async lower urgency | All categories share the same strict latency need |
| **Rollout** | Shadow deployment first | The change is low-risk and already well validated |

DESIGN DECISION

**Final case decision**

**Chosen design:** _____
**Accepted trade-off:** _____
**Evidence still needed:** _____
**Condition that changes the design:** _____

### Interview closing statement

INTERVIEW-READY ANSWER

## Trace every component to a requirement

I would begin with a credible shared multi-label or late-fusion baseline, measure cross-modal and category-specific failure cases, and move to the early-fusion multi-task design only when the added granularity and quality justify the training and serving complexity.